Empirical Software Engineering (2020) 25:1905–1950
https://doi.org/10.1007/s10664-020-09802-1

Boosting crash-inducing change localization with rank-performance-based feature subset selection

Zhaoqiang Guo[1] · Yanhui Li[1] · Wanwangying Ma[1] · Yuming Zhou[1] · Hongmin Lu[1] · Lin Chen[1] · Baowen Xu[1]

Published online: 2 March 2020
© Springer Science+Business Media, LLC, part of Springer Nature 2020

Abstract
Given a bucket of crash reports, it would be helpful for developers to find and fix the corresponding defects if the crash-inducing software changes can be automatically located. Recently, an approach called ChangeLocator was proposed, which used ten change-level features to train a supervised model based on the data from the historical fixed crashes. It was reported that ChangeLocator achieved a good performance in terms of Recall@1, MAP, and MRR, when all the ten features were combined together. However, in ChangeLocator, the redundancy between features are neglected, which may degrade the localization effectiveness. In this paper, we propose an improved approach ChangeRanker with a rank-performance-based feature selection technology (Rfs) to boost the effectiveness of crash-inducing change localization. Our experimental results on NetBeans show that ChangeRanker can achieve an improvement of 35.9%, 17.4%, and 15.3% over ChangeLocator in terms of Recall@1, MRR, and MAP, respectively. Furthermore, compared with three popular feature selection approaches, Rfs is able to select more informative features to boost localization effectiveness. In order to assess the real generalization capability of the proposed extension, we adapt ChangeRanker and ChangeLocator to locate bug-inducing changes on three additional data sets. Again, we observe that, on average, ChangeRanker achieves an improvement of 115.3%, 37.6%, and 41.2% in terms of Recall@1, MRR, and MAP, respectively. This indicates that our proposed rank-performance-based feature selection method has a good generalization capability. In summary, our work provides an easy-to-use approach to boosting the performance of the state-of-the-art crash-inducing change localization approach.

Keywords Crash-inducing change · Software crash · Bug localization · Feature subset selection · Crash stack

Guest Editor: Martin Monperrus

✉ Yuming Zhou
zhouyuming@nju.edu.cn

Extended author information available on the last page of the article

1 Introduction

It is not uncommon to see that software crashes occur in a software system, which brings a bad user experience or even unexpected disasters to users. When a crash occurs, there is an urgent need to find the corresponding defects in the software system. To facilitate finding the defects, many companies such as Microsoft (Kinshumann et al. 2011) and Google have deployed crash reporting systems to automatically send crash reports from end users to a centralized database. Typically, a crash report summarizes the state of the system and its environment at the time of crash. In the centralized database, similar crash reports are often grouped into clusters (buckets) (Dang et al. 2012). Ideally, each bucket consists of crashes relating only to one defect in the system.

Given a bucket of crash reports, it would be helpful for developers to find and fix the corresponding defects if the crash-inducing software changes (Moser et al. 2009) can be located. As highlighted in (Wen et al. 2016), the reasons are four-folds. First, we could find the committer of the crash-inducing changes, who is very likely to be the right developer to fix the defects. Second, the modified code in the crash-inducing changes could provide the contextual information for fixing the defects. Third, we could temporally circumvent the crash by reverting crash-inducing changes. Fourth, this will help for automatic program repair techniques to reduce the search space of defect fix location. In practice, it is time-consuming to manually locate the crash-inducing software changes.

In order to tackle the above problem, Wu et al. recently proposed ChangeLocator (Wu et al. 2018) to automatically locate crash-inducing changes for the given buckets of crash reports. First, ChangeLocator trains a supervised model using the data extracted from the buckets of crash reports that have been fixed. Then, for a given new crash bucket, ChangeLocator uses the trained model to compute the suspicious score for each candidate crash-inducing change extracted from the crash stack traces. After that, all the candidate crash-inducing changes are ranked in descending order from the most to the least suspicious. Finally, developers can simply select from the ranked list as many candidate crash-inducing changes to inspect as available resources will allow. Wu et al. used six release versions of NetBeans, a large-scale open-source software, to evaluate the effectiveness of ChangeLocator. In particular, Locus (Wen et al. 2016), a state-of-the-art IR-based bug localization approach (Rahman 2016), was adapted as the baseline approach to investigate the effectiveness of ChangeLocator. Their experimental results showed that ChangeLocator substantially outperformed the baseline approach Locus, regardless of whether the commonly used performance indicator Recall@1, MAP, or MRR is considered.

As described in (Wu et al. 2018), Wu et al. proposed ten candidate change-level features for building the supervised model in ChangeLocator. In order to determine which features should be used, Wu et al. first ran ChangeLocator using a single feature each time separately. In their experiment, logistic regression was the default modeling technique for building the supervised model, while MAP and MRR were the indicators to evaluate the effectiveness of ChangeLocator. As a result, they found that each single feature had a contribution to ChangeLocator. Then, they ran ChangeLocator by incrementally adding each of the ten features based on the performance of each single feature, from the worst feature to the best feature. Consequently, they observed that the best performance was achieved when all the ten features were combined together. Therefore, they concluded that all the ten features should be used in ChangeLocator.

However, the above analysis process only took into account the relevancy of features to the effectiveness of ChangeLocator but neglected the potential redundancy between features (i.e. one

feature may be correlated with another feature). Existing studies show that redundant features have negative effects such as unreliable and unstable estimates of coefficients (Cohen et al. 2003), multicollinearity problem, low prediction effectiveness (Gao et al. 2012; Guyon and Elisseeff 2003; Rathore and Gupta 2014), and unnecessary feature collection cost (Lewis et al. 2013). To reduce these impacts, therefore, we propose an improved approach ChangeRanker with a rank-performance-based feature selection technology (Rfs) to boost the effectiveness of crash-inducing change localization. Rfs can select the most informative features by maximum their ranking performance evaluated by ranking indicators (e.g., MRR). To examine the effectiveness and generalization of ChangeRanker, we conduct the following experiments:

1) Based on NetBeans project, we analyze four different construct factors that may affect the performance of ChangeRanker. This is an important experiment since it can select the optimal configuration of ChangeRanker, which can give a suggestion for researchers and developers to apply ChangeRanker properly (**RQ1**);
2) We compare ChangeRanker (Rfs) with the state-of-the-art crash-inducing change localization approaches (the popular feature selection approaches). The experiment can illustrate the effectiveness of ChangeRanker (Rfs) (**RQ2 and RQ3**);
3) We analyze the two external factors that can affect the result of ChangeRanker. The experiment can enlighten researchers or developers to choose proper forms of change candidates and evaluate an approach more reasonably (**RQ4**);
4) We extract ten change-level features from three projects which contain defect-introducing changes collected by Wen et al. (Wen et al. 2016) and conduct additional experiment based on these projects. This experiment is to examine the generalization of ChangeRanker (Rfs) (Section 7).

Based on the data from NetBeans project, our experimental results show that the number of features selected by Rfs is less than half of the number of features used in ChangeLocator. In particular, on average, ChangeRanker achieves a Recall@1 of 0.564, an MAP of 0.610, and an MRR of 0.641 on NetBeans. These results respectively correspond to an improvement in Recall@1 of 35.9%, an improvement in MAP of 15.3%, and an improvement in MRR of 17.4% over ChangeLocator. According to the Cliff's delta, the magnitude of difference between ChangeRanker and ChangeLocator is large (i.e., $|\delta| \geq 0.474$) regardless of whether Recall@1, MRR, or MAP is considered. Furthermore, compared with three popular feature selection approaches, ChangeRanker also exhibits a large improvement in localization effectiveness. In order to assess the real generalization capability of the proposed extension, we adapt ChangeRanker and ChangeLocator to locate bug-inducing changes on three additional projects (i.e. AspectJ, JDT, and Tomcat). We observe that, on average, ChangeRanker achieves a Recall@1 0.323, an MAP of 0.384, and an MRR of 0.410, leading to an improvement of 115.3%, 37.6%, and 41.2% over ChangeLocator respectively. This indicates that our proposed rank-performance-based feature selection method has a good generalization capability. In summary, our work provides an improved crash-inducing change localization approach, easier to interpret and use compared with the state-of-the-art approach.

The rest of this paper is organized as follows. Section 2 introduces the background about software crash and the existing approaches to locating crash-inducing changes. Section 3 presents the motivation for our study. Section 4 presents the proposed ChangeRanker approach. Section 5 describes the experimental design. Section 6 reports the experimental results. Section 7 examine the generalization capability of the proposed extension. Section 8 discusses

our findings. Section 9 analyzes the threats to the validity of our study. Section 10 concludes the paper and outlines directions for future work.

2 Background

In this section, we first introduce the background on crash reporting systems and crash-inducing changes. Then, we describe Locus and ChangeLocator, two existing crash-inducing locating approaches.

2.1 Crash reporting system

The users of a software system will have a bad user experience if crashes often occur in the running system, as it may cause serious troubles (e.g., data error, property loss, or even life threat) to users. In order to help developers to diagnose and fix the defects causing the crashes, it is necessary to collect the crash reports from user end. A crash report typically includes data such as stack traces, type of crash, build id, trends, and versions of software (Wu et al. 2014). Currently, many large companies have deployed crash report systems to prompt users to send crash reports to a centralized database. For example, Microsoft deployed Windows Error Reporting (Kinshumann et al. 2011), Google deployed CrashPad,[1] and Apple deployed Crash Reporter.[2]

For a popular software, since there are a huge number of users, a large number of crash reports may be collected by the crash reporting system. In particular, many of these crash reports are duplicate, as they are caused by the same defects. In order to facilitate developers debugging, a crash reporting system in general groups crash reports into different clusters (buckets) according to their features (e.g. similar crash stack), each cluster consisting of similar crash reports (Dang et al. 2012). Ideally, each bucket consists of crashes that are caused by the same defect. The bucket mechanism helps reduce the effort to find and fix the corresponding defects (Williams and Spacco 2008).

2.2 Crash-inducing changes

As stated in (Wu et al. 2018), software crashes are one of the most severe manifestations of software defects, which need to be fixed quickly and efficiently. A crash-inducing change is a software change (i.e. a single or several consecutive commits in a given period of time) that introduces one or several defects, which will raise software crashes under specific conditions, into the source code in a software system (Wang et al. 2014). Identifying crash-inducing changes have important implications for understanding the factors causing crashes and finding the solution to fix the crashes (Kim et al. 2008, Kim et al. 2011). For example, by identifying the modified code of a crash-inducing change, we can know the contextual clues that can help explain and fix the defects. Furthermore, by identifying the committer of a crash-inducing change, we know who is familiar with the modified code (in most cases, the committer is the developer modifying the code (Wen et al. 2016)). In this sense, the committer is a potential

[1] CrashPad [online]. Available: https://chromium.googlesource.com/crashpad/crashpad/+/master/README.md
[2] Technical note tn2123: Crashreporter (2015) [online]. Available: developer.apple.com/library/mac/#technotes/tn2004/tn2123.html

developer who can fix the corresponding defect. In particular, by reverting a crash-inducing change, we can provide a temporary solution to circumvent the crash locally and allow the software system to continue working without waiting for a fix.

2.3 Locus

Locus (Wen et al. 2016) is a state-of-the-art IR (information retrieval) based approach (Rahman et al. 2016; Youm et al. 2016; Rao and Kak 2011; Lukins et al. 2008; Moreno et al. 2013; Uneno et al. 2016) (different from spectrum based bug localization approach (Zhang et al. 2011; Le et al. 2015; Hoang et al. 2018)) that can locate defect-inducing changes. For a given bug report, Locus first computes the suspicious scores for the corresponding candidate defect-inducing changes (i.e. those software changes committed before the bug was reported). Then, all candidate defect-inducing changes are ranked in descending order from the most to the least suspicious. The suspicious score of a software change synthesizes the following three components: the similarity of natural language tokens between the bug report and change hunks, the similarity of code entity names between the bug report and change hunks, and the defect-proneness of the change indicated by the closeness between the bug report time and the commit time. In particular, in the former two components, the similarity is computed using the classical Vector Space Model (VSM) (Wang et al. 2014). Since Locus does not employ any historical label information to rank candidate defect-inducing changes, it is an unsupervised approach.

Wu et al. adopted Locus to locate crash-inducing changes (Wen et al. 2016). To this end, they first generated a bug report from a given crash bucket by using its crash stacks as the content of the bug report. Then, the generated bug report is fed to Locus to produce a list of ranked candidate crash-inducing changes. After manually inspecting the real bug reports corresponding to the crash buckets in NetBeans, they found that the bug description of most crashing bug reports included only crash stacks or crash reports. Therefore, they believed that it was reasonable to use the generated bug reports as the input of Locus in order to locate crash-inducing changes. In their study, Locus was used as the baseline approach to investigate the effectiveness of ChangeLocator, their proposed novel approach to locating crash-inducing changes given only buckets of crash reports.

2.4 ChangeLocator

Although the adopted Locus can be used to locate crash-inducing changes, Wu et al. found that its prediction performance was not satisfactory (Wu et al. 2018). To this end, they proposed ChangeLocator to locate crash-inducing changes. Unlike Locus, ChangeLocator is a supervised approach (Kim et al. 2007). In other words, ChangeLocator uses historical crash data to train a model before applying it to locate crash-inducing changes. The training data set is generated from the historical crash buckets that have been fixed. In the training data set, each instance (i.e. a candidate crash-inducing change derived from the crash reports in the historical crash buckets) consists of: (1) 10 change-level features (as shown in Table 1); and (2) a label indicating whether it is a crash-inducing change or not (identified by an improved SZZ algorithm (Jacek et al. 2005, Costa et al. 2017)). As can be seen, these 10 features can be classified into four dimensions: distance, diffusion, influence, and size. The distance dimension measures the inverse distance in space or time, the diffusion dimension measures the distribution over buckets, the influence dimension measures the affected range, and the size

dimension measures the change size. Since the number of the crash-inducing changes is much smaller than the number of non-crash-inducing changes, the training data set is typically imbalanced. ChangeLocator first applies a random under-sampling technique (Holmes et al. 2002) to obtain an imbalanced data set and then uses it to train a supervised model (the default modeling technique is logistic regression). After that, for a new crash bucket, ChangeLocator uses the following steps to locate the crash-inducing changes: (1) generate the candidate crash-inducing changes by analyzing the crash reports in the new crash bucket; (2) extract the same 10 change-level features for the candidate changes as used in the training data set; (3) use the trained model to compute for each candidate change the probability of being crash-inducing; and (4) rank the candidate changes in descendant order according to the predicted probability. Note that all the 10 features in the training and testing data sets are normalized to a range between 0 and 1 by a min-max normalization. After obtaining such a rank list, developers can simply select from the list as many candidate changes to examine as available resources will allow. If the rank is accurate, the crash-inducing changes will be found quickly.

For each crash report in the bucket, Wu et al. derived three forms of candidate crash-inducing changes. The first form of candidates (i.e. Form1) consists of all the revisions that introduce lines to the crashing methods recorded in the crash stack (Bettenburg et al. 2008). However, they noted that Form 1 may contain irrelevant candidates. The reason is that, in a crashing method, if the crashing line is not control/data dependent on the lines introduced by a revision, the revision is unlikely to be a crash-inducing change. As such, Wu et al. proposed the other two forms of change candidates: (1) reduce Form 1 to Form 2 by using control flow analysis as (Wu et al. 2014) to exclude irrelevant changes; and (2) reduce Form 1 to Form 3 by using backward slicing (Venkatesh 1991) to exclude irrelevant changes. Since backward slicing makes use of both control flow analysis and data flow analysis, Form 3 is indeed a subset of Form 2. After obtaining the crash-inducing candidates from each crash report, they are merged to obtain the candidate changes for the bucket. By default, ChangeLocator uses the candidate changes of Form 3.

Wu et al. used NetBeans, a popular open-source IDE,[3] as the subject project to evaluate the effectiveness of ChangeLocator. In the experiment, Wu et al. collected the crash-inducing data sets from seven versions of NetBeans. In order to locate the crash-inducing changes for a target version, they first combined the crash-inducing data sets from all previous versions as the training data set. Then, they used the rebalanced training data to train a logistic regression model. After that, the trained model was used to produce a rank of candidate crash-inducing changes for the target version. Consequently, Wu et al. reported that, on average, ChangeLocator can locate the crash-inducing changes for 44.7%, 68.5%, and 74.5% of the crash buckets by examining only top 1, 5, and 10 changes in the recommended, respectively. However, under the same settings, Locus only achieved 9.3%, 34.9%, and 54.2%, respectively. Therefore, Wu et al. concluded that ChangeLocator was much better than the existing state-of-the-art approach.

3 Motivation

In this section, we introduce our motivation of applying feature selection in terms of necessity and possibility.

[3] NetBeans project [online]. Available: https://netbeans.org/

Table 1 Change-level features proposed by Wu et al. (c is a candidate crash-inducing change and B is the corresponding crash bucket)

Dimension	Feature	Definition	Rationale
Distance	IADCP (Inverse Average Distance to Crashing Point)	$IADCP(c) = \frac{1}{1+\sum_{j=1}^{n}\frac{DCP_j(c)}{n}}$	*A change closer to the crash point (the top crash line in the crash stack trace) is more likely to be the crash – inducing change* n: number of crash in bucket B that contain change c *DCP j(c)*: the minimum distance between c and the crash point in the jth crash report of B.
	IADCL (Inverse Average Distance to Crashing Line)	$IADCP(c) = \frac{1}{1+\min_{j=1\ldots n} DCP_j(c)}$	*A change closer to the crash lines is more likely to be the crash- inducing change* DCL *j(c)*: the minimum distance between c and any crash line in the jth crash report of B
	ITDCR (Inverse Time Distance to Crash Revision)	$ITDCR(c) = \frac{1}{1+Rank(c)}$	*A change closer (in the committed time) to the revision the crash occurred is more likely to be the crash inducing change* Rank *(c)*: the rank of c among all the candidate changes based on the inverse committed time
Diffusion	RF (Revision Frequency)	$RF(c) = \frac{N_c}{N}$	*A change appearing frequently in the crash reports of a bucket is more likely to be the crash-inducing change* Nc: the number of crash reports in B that $\boldsymbol{c}$ appears N: the number of crash reports in B.
	IBF(Inverse Bucket Frequency)	$IBF(c) = \log\left(\frac{\#B}{\#B_c}+1\right)$	*A change appearing multiple buckets is less likely to be the crash-inducing change* #B: the number of buckets #Bc: the number of buckets whose changes include c.
Influence	CC (Crash Component)	$CC(c) = \begin{cases} 1 & c?B = 1 \\ 0 & c?B = 0 \end{cases}$	*A change affects the source files in the component that B belongs to is more likely to be the crash-inducing change* c?B: 1 if c affects the source files in the component that B belongs to; otherwise,0
	NAF(Number of Affected Files)	*NAF(c)* = the number of affected files by a change c	*A change affects more files is more likely to be crash-inducing change*
Size	RLOAC(Revision's Lines of Added Code)	*RLOAC(c)* = the number of lines of added code by a change c	*A change adds more code is more likely to be crash-inducing change*
	RLODC(Revision's Lines of Deleted Code)	*RLODC(C)* = the number of lines of deleted code by a change c	*A change deletes more code is more likely to be crash-inducing change*
	RLOCC	*RLOCC(c)* = the number of lines of modified code by a change c	*A change modifies more code is more likely to be crash-inducing change*

3.1 The necessity of boosting ChangeLocator with feature selection

As can be seen, ChangeLocator achieves a good crash-inducing change localization performance, which depends on the quality of ten change-level features. However, their feature analysis process neglected the potential redundancy between features (i.e. one feature may be correlated with another feature). Indeed, dealing with redundant features plays an important role in practice. First, the phenomena of multicollinearity in redundant features may lead to unreliable and unstable estimates of coefficients in logistic regression models used in ChangeLocator (Cohen et al. 2003). Second, many studies show that eliminating redundant features not only helps for correcting the multicollinearity problem but also helps for improving the prediction effectiveness (Gao et al. 2012; Guyon and Elisseeff 2003; Rathore and Gupta 2014). Third, dropping redundant features can make model building and application more efficient in practice, because this can save unnecessary feature collection cost. This is especially important, as a prerequisite for deploying a defect prediction model in a large company is that it must be able to scale to large software repositories (Lewis et al. 2013). Therefore, there is a need to conduct feature selection to remove redundant features in ChangeLocator.

Since feature selection is an important preprocessing step, a great number of studies (Ghotra et al. 2017; Hall and Holmes 2003; Rodríguez et al. 2007; Xu et al. 2016; Rathore and Gupta 2014; Muthukumaran et al. 2015) have investigated the impact of feature selection technologies to prediction models, which show the importance and necessity of feature selection. According to the result of Rodriguez et al.'s study (Rodríguez et al. 2007), the dataset with fewer features (attributes) can maintain or even improve the prediction capability of a model. Gao et al. (Gao et al. 2012) and Guyon et al. (Guyon and Elisseeff 2003) pointed out that redundant, correlated, or irrelevant features can hinder the model performance, so it is necessary to remove these features. According to Xu et al.'s (Xu et al. 2016) experiment results on two versions of NASA dataset, different feature selection technologies affect the defect prediction performance to varying degrees.

Many researchers (Kohavi and John 1998; Kohavi 1995; Hall and Holmes 2003) proposed different feature selection technologies to improve the performance of models. In summary, there are three kinds of feature selection methods widely used in the literature.

Wrapper-based subset selection methods In the literature, wrapper-based methods are usually believed as the best feature selection methods (Kohavi and John 1998). They evaluate feature sets by using a specific learning technique (e.g., *Logistic*). It uses cross-validation (Kohavi 1995) to estimate the performance of the learning technique for a subset of features and selects an optimal feature subset according to the performance scores.

Filter-based subset selection methods They consist of correlation-based and consistency-based subset selection methods. The former aims to find features with low inter-correlation, but high correlation with the class label. However, the latter uses consistency as an indicator to measure the importance of a feature subset (Hall and Holmes 2003). To this end, different search methods (e.g., BestFirst, Rank Search) can be applied to search the subset.

Filter-based feature ranking methods In a ranking method, features are ranked in descendant according to the evaluation scores. The score of each feature can be obtained by analyzing metric statistics (e.g., Chi-Square), importance probability (e.g., InfoGain and GainRatio),

sample instances (Hall and Holmes 2003) (e.g., ReliefF), and classifiers (e.g., SVM and OneR).

The above numerous studies demonstrate that feature selection is an important issue that deserves to pay more attention in model building. In particular, there is no feature selection for ChangeLocator. In order to boost the performance of ChangeLocator, therefore, it is necessary to explore proper feature selection technology based on the ten change-level features.

3.2 The possibility to boost ChangeLocator with feature selection

In this subsection, we present an example to illustrate the possibility to boost ChangeLocator with feature selection. NetBeans is a popular open-source IDE that can be used to quickly and easily develop desktop, mobile, and web applications with many programming languages such as Java, JavaScript, HTML5, PHP, and C/C++. According to Wu et al.'s study, NetBeans 6.5 contains 25,558 files, 5524 KLOC, and 110,887 revisions. The crash reports are grouped into 42 crash buckets. In total, there are 1158 candidate crash-inducing changes in 42 buckets. For each candidate change, Wu et al. extracted 10 change-level features (as described in Table 1) and a label indicating whether it was a crash-inducing change. On average, each bucket has 27.57 candidate changes. For 23 of the 42 buckets, the crash-inducing changes are in the candidate change set. In these 23 buckets, the total number of candidate changes are 644, of which 26 are crash-inducing changes and 618 are non-crash-inducing changes. For the simplicity of presentation, we will: (1) use "hit set" to denote the data set combined from those buckets whose candidate changes include at least one crash-inducing change; and (2) use "full set" to the data set combined from all buckets. Therefore, for NetBeans 6.5, the "hit set" consists of 644 instances, while the "full set" consists of 1121 instances.

Figure 1 reports the distribution of the values of each feature NetBeans 6.5 "hit set". For each feature, there are two boxplots. The left boxplot reports the distribution on the crash-inducing changes (i.e. "TRUE"), while the right boxplot reports the distribution on the non-crash-inducing changes (i.e. "FALSE"). In particular, we use the Cliff's δ to characterize the difference of the values between the left boxplot and the right boxplot. As can be seen, CC and IADCP have the largest two δ values. Therefore, of all the 10 change-level features, CC and IADCP have the strongest ability to distinguish between the crash-inducing changes and the non-crash-inducing changes. Furthermore, we can see that all the crash-inducing changes have a CC value of 1 (see the left boxplot of CC). This suggests that, on the NetBeans 6.5 "hit set", it appears that "a CC value of 1" is a necessary condition of a candidate change being a crash-inducing change. In other words, if a candidate change has a CC value of 0, it is not a crash-inducing change in NetBeans 6.5.

In order to explore the predictive ability of IADCP, we sort the 644 changes in descendant order according to the value of IADCP. Consequently, we find that the top 26 candidate changes have the maximum IADCP value of 1. Of these 26 changes, 10 changes are crash-inducing changes (i.e. the precision is 10/26). When looking at the CC value, 20 changes have a CC value of 1, while 6 changes have a CC value of 0. Since "a CC value of 1" is a necessary condition of a candidate change being a crash-inducing change, this raises an interesting question: what if we use the combination of IADCP and CC? As such, we rank the 644 changes in descendant order according to the value of IADCP × CC. Consequently, we find that, the top 20 candidate changes have a value of 1, in which 10 changes are the crash-inducing changes. The above observation reveals that, compared with IADCP, IADCP × CC

improves the ability to rank the candidate changes (as the precision increases from 10/26 to 10/20).

Inspired by this fact, we realize that a simple feature combination may have a good ability to rank candidate changes, which shows the possibility to boost ChangeLocator with feature selection. As such, the following question naturally raises: how to find a simple yet effective feature subset? Before answering this question, we need to address the following two sub-questions: (1) "which features should be used in the subset?" and (2) "how to construct the ranking model?". We answer the above two sub-questions in the next section.

4 Approach

In this section, we present ChangeRanker, a supervised approach to locating crash-inducing changes based on a ranking-performance based feature subset selection. First, we describe the framework of ChangeRanker (we use the candidate crash-inducing changes of Form3 by default). Then, we present Rfs, a ranking-performance based approach to selecting an informative feature subset. Finally, we introduce how to use the selected feature subset to build a supervised prediction model for ranking candidate crash-inducing changes.

4.1 Approach overview

Figure 2 describes the overall framework of ChangeRanker. For the simplicity of presentation, we assume that a project has *k* versions. Of these versions, the former *k*-1 versions (i.e. version 1, version 2, …, version *k*-1) are historical versions, in which crash buckets have been resolved (i.e. we know which changes are crash-inducing changes for each bucket). The *k*th version is the target version, in which there is a target unresolved bucket. The problem we face is how to locate crash-inducing changes for the unresolved target bucket. At a high level, ChangeRanker involves the following three phases: data preparation, model training, and model testing. At the data preparation phase, preprocess the training data collected from historical versions to make

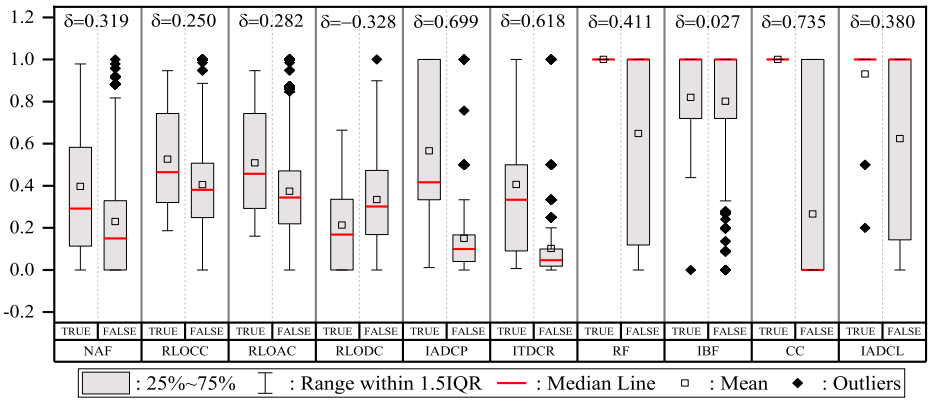


Fig. 1 The distribution of each feature over the crash-inducing changes and non-crash-inducing changes on NetBeans 6.5 (TRUE: crash-inducing changes, FALSE: non-crash-inducing changes)

them appropriate for building a supervised model for the target bucket. At the model training phase, use a supervised modeling technique to build a prediction model. At the model testing phase, apply the model to rank crash-inducing changes and evaluate its ranking performance. By the above phases, it is expected that the knowledge about the effect of change-level features on crash-inducing can be learned from the resolved buckets of historical versions by the supervised model to predict crash-inducing changes in the unresolved target bucket of the target version. More specifically, the application of ChangeRanker in practice consists of the following steps:

Step 1 extract candidate training data. From each resolved bucket of the historical versions, first identify the candidate crash-inducing changes. Then, for each candidate crash-inducing change, extract m change-level features ($m = 10$ by default, i.e. the 10 change-level features shown in Table 1) and a label indicating whether it is an actual crash-inducing change.

Step 2 combine candidate training data. This step simply combines the candidate training data from k-1 historical versions to obtain the combined training data. The purpose is to make full use of available historical data for model building.

Step 3 handle class-imbalanced training data. This step uses a random under-sampling technique to obtain a class balanced training data, in which the number of crash-inducing changes is similar to the number of not crash-inducing changes.

Step 4 select informative features. This step uses Rfs (ranking-performance-based feature selection) to select features that are not redundant and have a strong predictive capability. Only these selected features will be considered for model building.

Step 5 train a supervised model. This step uses logistic regression to build a supervised model to capture the relationship between the features (i.e. the independent variables) and crash-inducing changes (i.e. the dependent variable) in the training data.

Step 6 extract test (feature) data. This step extracts those informative features identified in step 4 from the target unresolved bucket. In other words, there is no need to extract all the 10 change-level features proposed in Wu et al.' study as the test (feature) data.

Step 7 apply the supervised model. The step applies the trained model to predict the suspicious degree of being crash-inducing change for the candidate changes in the target bucket. For each change, the predicted suspicious degree is obtained by introducing the corresponding test (feature) data to the model.

Step 8 evaluate the ranking performance. This step outputs a ranked list of all candidate changes from the most to the least predicted suspicious degree. Under this scenario, the performance report is generated by comparing the predicted suspicious degree with the test (oracle) data.

The above steps show the framework of ChangeRanker, which is similar to ChangeLocator. However, there is a distinct difference between them. ChangeRanker employs Rfs (ranking-performance-based feature selection) to select informative features from the training data before training a model (step 4). At the same time, only the same informative features will be extracted in the target unresolved bucket to apply the trained model (step 6). The use of Rfs technology can exclude the influence of uninformative features to the ranking performance and hence improve the usefulness of the prediction model in locating crash-inducing changes.

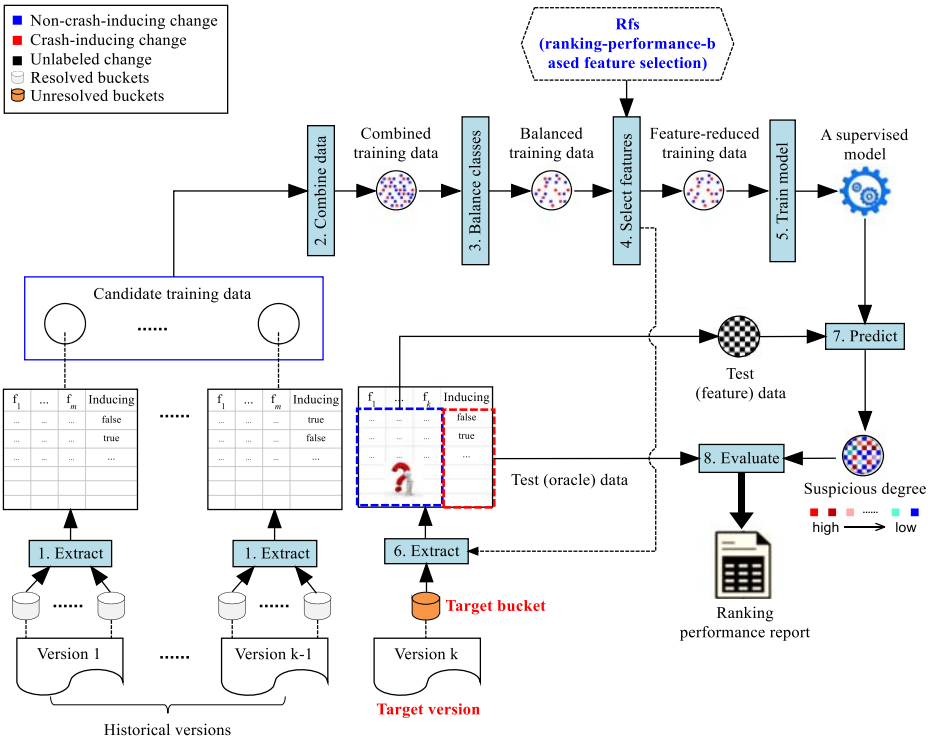


Fig. 2 The overall framework of ChangeRanker

4.2 Rfs: Ranking-performance-based feature subset selection

As mentioned above, Wu et al.'s study did not take into account the redundancy between change-level features, which may have a negative influence on the localization of crash-inducing changes. In order to decide which features should be used for model building, we propose a new feature subset selection technology Rfs (Ranking-performance-based feature subset selection) to select a feature subset (informative features). Specifically, Rfs is designed for selecting an informative feature subset that can lead to a good prediction model with a high ranking performance. To this end, Rfs uses a ranking performance indicator (MRR by default) to guide the selection of change-level features.

Suppose there are n features, the objective of Rfs is to select t $(1 \le t \le n)$ features to maximize the ranking performance value. Mathematically, it can be represented as follows:

$$\begin{aligned} &\max RankingPerformance(f_1x_1, f_2x_2, \ldots, f_nx_n) \\ &s.t. x_i \in \{0, 1\}, \sum_{i=1}^{n} x_i = t \end{aligned} \tag{1}$$

where f_i denotes the ith feature in the full set of features, $x_i = 1$(or 0) indicates that the feature f_i is selected (or not selected), t denotes the number of features we selected, $RankingPerformance(f_1x_1, f_2x_2, \ldots, f_nx_n)$ denotes the ranking performance (MRR by default) that a feature subset (including t features) can achieve.

The optimization in (1) is a typical 0–1 integer programming problem. The simplest solution is to conduct an exhaustive search to find the optimal feature subset. However, given a set of change-level features, it may be computationally-intensive to search the space of all

Algorithm 1: *Explore(T, S)*

```
// Built a feature combination tree
Input: T is a reference to the tree root
       S is a candidate feature set {s_1, s_2, …, s_n}
Output: Void
1: if S = ∅ || T.length == MaxDepth then   // This is a leaf node or path length is equal to the maximum depth
2:    return                               // terminate this
3: end
4: foreach s_i ∈ S do
5:    S_used ← T.S_used ∪ s_i              // Update the current used feature set
6:    P_o ←T.performance                   // Get the old performance
7:    P_n ← getPerformance(S_used, op, indicator)  //Calculate the new performance indicator score on
                                                   // feature subset combined by operator op
8:    if P_n >= P_o then                   // Need to prune the branches?
9:       Create a child node N_c           // record the used feature set and the corresponding performance
10:      Explore(N_c, S − s_i)             // Continue to grow in this path
11:   end
```

Fig. 3 A greedy strategy based feature combination tree generation algorithm

feature combinations. When exploring candidate feature subset, therefore, Rfs uses a greedy strategy (Nguyen et al. 2011) to construct a feature combination tree to reduce the search space. Figure 3 describes the algorithm of Rfs whose input is a set of change-level features and whose output is a pruned feature combination tree. As can be seen, algorithm 1 proceeds in a recursive manner. The growth process starts with a tree root node using depth first as the growth strategy. For each candidate feature s_i in S, add it to the used features set and calculate the new performance score based on all features in S_{used} successively. It is worth mentioning that the performance score is calculated according to the actual situation that can be defined by users (line 7). In our study, we use "×" as the default operator and MRR as the default performance indicator. When the new performance value is higher than the original performance value, continue to explore in this path until the performance value no longer grows or the path length is equal to the maximum tree depth specified by users (line1, line 8 to 11). After the exploration is completed, the features within every path from the root node to each leaf node form a locally better combination. By sorting the performance values from the largest to the smallest, we can find the best feature combination(s).

To enable Rfs to select the optimal feature subset, proper parameters should be set for Rfs. More specifically, the following three parameters should be addressed:

- **Feature combination operator.** As mentioned in Section 3, we manually analyze the distributions of IADCP and CC. We find that IADCP × CC has a better ranking ability than IADCP. In particular, IADCP × CC has a clear intuitive interpretation: when a candidate change does not affect the source files in the components that the bucket belongs to (i.e. CC = 0), the change has a very low possibility of being a crash-inducing change even if it is very close to the crash point. Furthermore, the operator "×" has a low computation cost. Based on this observation, given a set of features, we decide to use the "×" operator to generate the feature combination by default. Indeed, there are other operators such as '+' also can be used for feature combination. We will discuss the influence of operators in later.

- **Maximum depth of feature combination tree.** According to the motivation example in Section 3, we find that only a few features (IADCP, CC etc.) may be useful in locating crash-inducing changes. In addition, a large number of features may be redundant so that is not necessary to retain them. The maximum depth of feature combination tree denotes the maximum feature number in an exploration path (feature subset). In order to select effective features more accurately and exclude most of the useless features, we set the default value of maximum depth to 3, which accounts for only 30% of all features. Other value can also be used in practical exploration process.
- **Performance indicator.** In the ranking scenario, MRR (Mean Reciprocal Rank) is one of the most commonly used performance indicators. Formally, MRR has the following definition:

$$MRR = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{rank_i} \tag{2}$$

Here, $|Q|$ denotes the number of crash buckets and $rank_i$ is the rank position of the first crash-inducing change for the ith bucket. MRR is the average of the reciprocal ranks of results over the $|Q|$ buckets. The range of MRR varies from 0 to 1. The higher the MRR value, the better the ranking performance of a combination. As can be seen, MRR only cares about the single highest-ranked crash-inducing change in each bucket. In our context, it is important for developers to find the first crash-inducing change as soon as possible (Manning et al. 2008). In this sense, MRR can be considered as an appropriate performance indicator to guide the search of a simple yet effective feature combination. There are other representative performance indicators (e.g., MAP) can be used as substitution.

To better demonstrate the feature subset exploration process, we apply Rfs to select feature subset in NetBeans 6.5 as an example. Figure 4 shows the top 10 feature subsets (paths) in the feature combination tree after exploration. In this figure, the pink circle node is the *null* root node, while each ellipse node denotes a feature in one path. In particular, each green square node stores the MRR value for the corresponding path. For example, the left-bottom green square node has an MRR value of 0.413. This means that, if we rank the candidate changes in descendant order according to the value of the combination RLOAC × IADCP × CC, the resulting MRR on the 23 crash buckets will be 0.413. In order to facilitate observing the best path(s), we use a red square node to show the path consisting of the best features. Furthermore, the ellipse nodes in the path corresponding to the best combination will be shown in cyan color. From Fig. 4, we can see that the best combination (feature subset) is IADCP × CC × IADCL, which has an MRR value of 0.443.

Table 2 summarizes the number of occurrences of each feature on the paths in Fig. 4. As can be seen, the top two frequency features are IADCP and CC, which occurs 9 times and 8 times respectively. This indicates that the two features are very important for improving the performance of MRR. Based on the above observations, we deem that IADCP × CC × IADCL should be the simple superior combination based on the train set 6.5 we want to find. Meanwhile, the three features form the superior informative feature set.

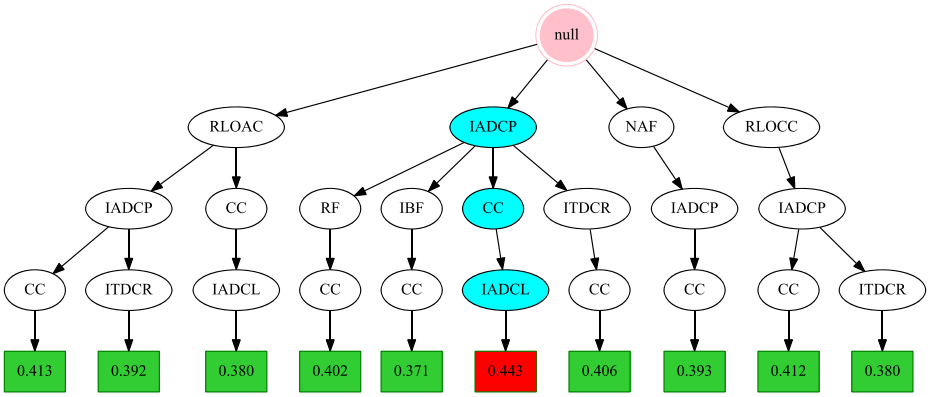


Fig. 4 Top-10 feature subsets in the feature combinations tree based on the MRR

4.3 Training a supervised ranking model

For the simplicity of presentation, we use a feature vector $\boldsymbol{x} = [x_1, x_2, \ldots, x_n]^T$ to represent the selected feature subset by Rfs, where x_i denotes the ith feature in the subset. After that, we train a ranking model using Logistic Regression (Allison 1999) (*Logistic*) based on the feature subset by the following formula.

$$p(\boldsymbol{x}) = \frac{1}{1 + e^{-(\boldsymbol{w}^T\boldsymbol{x}+b)}} \tag{3}$$

Here, $\boldsymbol{w} = [w_1, w_2, \ldots, w_n]^T$ is a weight vector and b is one additional parameter needed to be fit on the training data. As can be seen, $p(\boldsymbol{x})$ has a value between 0 and 1. In our context, $p(\boldsymbol{x})$ represents the probability of a change being a crash-inducing change. We use feature vectors extracted from historical data to fit the logistic regression model. For a new given bucket, we extract a feature vector for each change candidates and input them into the logistic regression model. We can get a suspicious value for each change. After that, all changes are ranked in descending order from the most to the least suspicious.

5 Experimental design

In this section, we introduce the experimental design to investigate the effectiveness of ChangeRanker in locating crash-inducing changes, including the data sets, performance indicators, statistical tests, and research questions.

Table 2 The frequency of features in the top10 combinations

Feature Name	NAF	RLOCC	RLOAC	RLODC	IADCP	ITDCR	RF	IBF	CC	IADCL
Frequency	1	2	3	0	**9**	3	1	1	**8**	2

5.1 Data sets

We use the same NetBeans data sets used by Wu et al. (Wu et al. 2018) to evaluate the effectiveness of ChangeRanker. This enables us to directly compare whether ChangeRanker is superior or inferior to ChangeLocator in locating crash-inducing changes. In Wu et al.'s study, the data sets were collected from the following seven versions: 6.5, 6.7, 6.8, 6.9, 7.0, 7.1, and 7.2. In their study, NetBeans 6.7, 6.8, 6.9, 7.0, 7.1, and 7.2 were used as the test data sets and previous versions as training data sets. In order to make a fair comparison, our study also uses NetBeans 6.7, 6.8, 6.9, 7.0, 7.1, and 7.2 as the test data sets to evaluate the effectiveness of ChangeRanker.

Table 3 summarizes the information of the six NetBeans versions. The second to fourth columns report for each version the project size in files, source lines of code, and revisions, respectively. The fifth column reports the number of crash buckets. In the sixth column, #oracle denotes the number of crash buckets that have an oracle (i.e. the crash-inducing changes are in the candidate change set of Form 3). #c < =5 denotes the number of crash buckets having an oracle whose candidate change set has a size less than or equal to 5. #c < =10 has a meaning similar to #c < =5, but with a different threshold 10. The seventh and eight columns report the number of candidate changes in the maximum and minimum change sets, respectively. The ninth column reports the total number of candidate changes of Form3. The last column reports that the maximum number of crash buckets for which the crash-inducing changes can be located. For example, for NetBeans 6.7, the total number of crash buckets is 61, of which 46 crash buckets have an oracle. Therefore, based on the data set, 46 is the maximum possible number of crash buckets that a localization approach could locate the crash-inducing changes. As a result, the maximum recall is 0.754 = 46/61.

As shown in the fourth column, the number of revisions (i.e. all the possible candidate changes) for each version is huge. Therefore, it is time-consuming and impractical to manually inspect all the revisions for locating crash-inducing changes. This is the reason why there is a need to use an automatic approach to recommend a rank list of candidate changes. When looking at the ninth column, we can see that the number of candidate changes of Form3 is far lower than the number of revisions. This means a large effort-saving if we only inspect the candidate changes of Form3. In the ideal case, on average, we could locate the crash-inducing changes for 76.8% of crash buckets, as they are included in the candidate changes of Form3.

Table 3 The detail information on the evaluation subjects

Version	#files	KLOC	#Revs	#bucket	(#oracles /#c < =5/ #c < =10)	max change set size	min change set size	#chages of form 3	max recall
6.7	48,044	10,331	139,189	61	(46/5/6)	130	3	1323	0.754
6.8	49,918	10,717	158,936	52	(40/3/0)	105	2	1233	0.769
6.9	53,264	11,632	176,495	40	(27/57)	91	3	942	0.675
7.0	46,189	9745	196,886	38	(27/27)	83	3	777	0.711
7.1	38,900	8429	217,206	41	(36/1/5)	76	3	1190	0.878
7.2	39,678	8666	235,526	39	(32/3/6)	73	2	1328	0.821
total	275,993	59,520	1,124,238	271	(208/19/51)	130	2	6793	0.768

5.2 Performance indicators

Both ChangeLocator and ChangeRanker will produce a list of candidate changes ranked from the most to the least suspicious for introducing crashes. It would be better if crash-inducing changes are ranked higher in the ranked list. We use the following performance indicators to evaluate the rank effectiveness:

- Recall@k. This indicator measures the percentage of buckets whose corresponding crash-inducing changes are correctly recommended at least one time in the top k ranked results. Generally, since we are only considering the top few ranks, and only require finding one of the crash-inducing changes for each crash bucket, therefore the threshold k usually is set as 1, 5, and 10. The higher the value, the better the rank effectiveness. In our context, it would be better if crash-inducing changes could be found quickly. In this sense, Recall@1 is more important compared with Recall@5 and Recall@10.
- MRR (Mean Reciprocal Rank). MRR (Manning et al. 2008) measures the average performance of the first found crash-inducing change for each bucket in a project. In other words, the later crash-inducing changes, even if they are also important, will be neglected. See details in Section 3.3.
- MAP (Mean Average Precision). MAP (Manning et al. 2008) considers whether all of the crash-inducing changes tend to get ranked highly. Formally, MAP has the following definition:

$$MAP = \sum_{i=1}^{|Q|} \frac{AvgP(i)}{|Q|} \tag{4}$$

Here, $|Q|$ denotes the number of crash buckets and $AvgP(i)$ denotes the average precision of all crash-inducing changes in the ith crash bucket. Assume that K is the set of ranks of all crash-inducing changes for the ith crash bucket. Then, we have:

$$AvgP(i) = \sum_{k \in K} \frac{Prec@k}{|K|} \tag{5}$$

where $Prec(k)$ denotes the precision at the given cut-off rank k, i.e.,

$$Prec@k = \frac{\#of\ \ crash{-}inducing\ \ changes\ in\ top\ k}{k} \tag{6}$$

5.3 Statistical significance test

Given two sets of values of performance indicators, we need to conduct a statistical signification test to examine the difference of two approaches. In our experimental settings, we can get two sets of results based on each performance indicator. Therefore, our experimental results appear in pairs and hence we use Wilcoxon signed-rank test, which tests the null hypothesis that two related paired samples come from the same distribution, to calculate the p value. Note that, there is another test Mann-Whitney U test that can also be used to calculate p value. However, it is designed to compare the difference between two independent groups when the dependent variable is either ordinal or continuous, but not normally distributed. In other words,

it assumes that all the observations are independent of each other (no basis for pairing). Therefore, it is inappropriate for our experiment. After getting p value, we adjust p values by the BH-corrected (Benjamini-Hochberg corrected) (Benjamini and Hochberg 1995) to control for the False Discovery Rate (FDR).

We use Cliff's delta (δ) (Benavoli et al. 2016; Shaffer 1986; Wilcoxon 1945) to show the effect size that quantifies the amount of difference between two groups of observations beyond the p values interpretation. It can be understood as a useful complementary analysis for the corresponding hypothesis testing. The corresponding relationship between δ value and effect size level is shown as follows.

$$Effect\ level = \begin{cases} |\delta| < 0.147, & Negligible \\ 0.147 \leq |\delta| < 0.33, & Small \\ 0.33 \leq |\delta| < 0.474, & Medium \\ |\delta| \geq 0.474, & Large \end{cases} \tag{7}$$

5.4 Research questions

We want to identify whether the new approach, ChangeRanker, is able to locate crash-inducing changes effectively and even outperforms ChangeLocator (in our comparison, we use the exactly same data sets and experimental setup to make a fair comparison). To this end, we want to investigate the following research questions:

5.5 RQ1: How do the configuration factors affect the effectiveness of ChangeRanker?

The motivation of this RQ is to get the optimal configuration of ChangeRanker so that it can achieve the best performance. In particular, we discuss four important factors, including modeling techniques, feature selection settings, maximum depth of feature combination tree, and class-imbalance strategies. According to (Wu et al. 2018), the experiments in this RQ are all conducted on the change candidates set with Form3.

(1) **The influence of modeling techniques**

Modeling technique is an important variable that can affect the prediction result of ChangeRanker just as ChangeLocator directly. According to existing work (Wu et al. 2018), we use Logistic Regression as our default modeling technique in our default settings. In this RQ, we compare the ranking performance of eight popular and representative modeling techniques, including Logistic Regression (*Logistic,*), Naive Bayes (*NB*), Mulit-Layer Perceptron (*MLP*), Decision Tree (*J48*), k-Nearest Neighbor (*IBk*), Random Forest (*RF*), Support Vector Machine (*SVM*), and PART Decision Tree (*PART*). All of these techniques are implemented on the top of the open-source data mining software WEKA. Note that, in the default implementation of WEKA, there are three layers (includes an input layer, a hidden layer, and an output layer) in the structure of MLP. The number of nodes of the input layer is the number of used features. The number of nodes of the hidden layer is set to (#feature + #output) / 2. Since the number of features is small (< 10) after applying Rfs, we use a hidden layer in the structure of MLP. Because there are two types for a change, the number of nodes of the output layer is set to 2. According to Tantithamhavorn et al. (Tantithamthavorn et al. 2019), the models with parameter optimization can improve the prediction performance. To obtain a reliable

comparison result, we compare the results outputted by each model with different parameters. Due to the large number of parameters, some classifiers can lead to a combinatorial explosion of parameter setting candidates. For instance, there are 6 numerical parameters for the MLP that implemented by Weka. There are 15,625 ($5 \times 5 \times 5 \times 5 \times 5 \times 5$) parameter settings needed to be tested when considering 5 candidate values for each parameter, which is inadvisable in the literature. Therefore, following in (Lin et al. 2018), we use the control variable method to select parameters. That is to say, we only change the value of one parameter while the others are set as the default values. In particular, we select 5 values (including the default value) as the candidates for each numeric parameter. Table 4 provides an overview of the examined parameters of the studied eight modeling techniques.

(2) **The influence of feature selection settings**

The feature selection setting of Rfs is also an important factor that may affect the effectiveness of ChangeRanker, which includes the selection of operator and performance indicator. The selection of operators determines how to combine a set of features. Meanwhile, the selection of performance indicator determines how to evaluate a feature subset. In Section 4, we use the default feature subset selection setting (i.e., "×" operator and MRR rank performance indicator) to find informative features. If we use other operators and performance indicators, can we still select a good feature set for constructing ChangeRanker? In order to explore the optimal feature selection setting, we test informative feature subsets using other available settings ("+" operator, Recall@1, and MAP rank performance indicators).

(3) **The influence of maximum depth of feature combination tree**

Another important factor is the depth of feature combination tree. According to the algorithm of Rfs, a set of features will be generated by adding new features into one path of the tree. The longest path determines the maximum depth of the feature combination tree. In other words, the depth of the tree determines the size of a feature subset that Rfs selected. To reduce combination search space and to select informative features more accurately (exclude more useless features), one is more inclined to use a smaller maximum depth. However, to remain the important features, a larger maximum depth is needed. In this subsection, therefore, we explore the appropriate value of maximum depth that can select more informative features as accurate as possible. In particular, we compare the performance results of ChangeRanker by setting different maximum depth values (from 1 to 10).

(4) **The influence of class-imbalance strategies**

The last important factor is the solution to class-imbalance strategies. According to (Wu et al. 2018), the balanced training data set can maximize the advantage of prediction models. The simplest solution to handle the class-imbalance problem in is US (i.e., under-sampling) approach, which is used in ChangeLocator. In fact, there are many other strategies that can deal with the class-imbalance problem. What if we use other class-imbalance strategies? To investigate the influence of class-imbalance strategies, we use SMOTE (i.e., synthetic minority over-sampling technique) (Chawla et al. 2002) and UBag (Under-Bagging) (Wang and Yao 2009) to rerun ChangeRanker. SMOTE is a popular sampling approach, while the UBag is the best-performing approach reported in a recent study (Song et al. 2018).

Table 4 The overview of the examined parameters of the studied eight modeling techniques

Classifie	Parameter option	Parameter Description	The default (bold font) and candidate parameter values
Logistic	-R < ridge>	[N] The ridge in the log-likelihood.	{1.0E-4, 1.0E-5, 1.0E-6, 1.0E-7, **1.0E-8**}
	-M < number>	[N] The maximum number of iterations. (default −1, until convergence)	{**−1**, 50, 100, 200, 500, 1000}
NB	-K	[B] Use kernel density estimator rather than normaldistribution for numeric attributes.	{true, **false**}
	-D	[B] Use supervised discretization to process numeric attributes.	{true, **false**}
MLP	-L < learning rate>	[N] Learning rate for the backpropagation algorithm.	{0.1, 0.2, **0.3**, 0.4, 0.5}
	-M < momentum>	[N] Momentum rate for the backpropagation algorithm.	{0.1, **0.2**, 0.3, 0.4, 0.5}
	-N < number>	[N] Number of epochs to train through.	{100, 300, **500**, 800, 1000}
	-V < percentage>	[N] Percentage size of validation set to use to terminate training.	{**0**, 5, 10, 15, 20}
	-S < seed>	[N] The value used to seed the random number generator.	{**0**, 1, 3, 5, 7, 9}
	-E < threshold>	[N] The number of consecutive increases of error allowed for validation testing before training terminates.	{5, 10, **20**, 30, 50}
	-C	[B] Normalizing a numeric class will NOT be done.	{true, **false**}
	-I	[B] Normalizing the attributes will NOT be done.	{true, **false**}
	-R	[B] Resetting the network will NOT be allowed.	{true, **false**}
	-D	[B] Learning rate decay will occur.	{true, **false**}
J48	-C < confidence>	[N] Set confidence threshold for pruning.	{0.1, **0.25**, 0.3, 0.4, 0.5}
	-M < minimum>	[N] Set minimum number of instances per leaf.	{1, **2**, 3, 4, 5}
	-R	[B] Use reduced error pruning.	{true, **false**}
	-N < number>	[N] Set number of folds for reduced error pruning. One-fold is used as pruning set.	{2, **3**, 4, 5, 10}
	-A	[B] Laplace smoothing for predicted probabilities.	{true, **false**}
	-Q < seed>	[N] Seed for random data shuffling.	{**1**, 3, 5, 7, 9}
	-S	[B] Do not perform subtree raising.	{true, **false**}
	-U	[B] Use unpruned tree.	{true, **false**}
IBk	-I	[B] Weight neighbors by the inverse of their distance.	{true, **false**}
	-K < neighbors>	[N] Number of nearest neighbors (k) used in classification.	{**1**, 3, 5, 7, 9}
	-E	Minimize mean squared error rather than mean absolute error.	{true, **false**}
	-W < number>	[N] Maximum number of training instances maintained. Training instances are dropped FIFO.	{**0**, 1, 2, 3, 4}
	-X	[B] Select the number of nearest neighbors between 1 and the k value specified using hold-one-out evaluation on the training data.	{true, **false**}
RF	-I < number>	[N] Number of trees to build.	{50, 80, **100**, 200, 500}
	-K < number>	[N] Number of features to consider.	{**0**, 1, 2, 3, 4}
	-S < seed>	[N] Seed for random number generator.	{**1**, 3, 5, 7, 9}

Table 4 (continued)

Classifie	Parameter option	Parameter Description	The default (bold font) and candidate parameter values
	-depth < depth>	[N] The maximum depth of the trees, 0 for unlimited.	{**0**, 1, 2, 3, 5}
SVM	-C < complexity>	[N] The complexity constant C.	{0.5, **1**, 2, 3, 4}
	-N < option>	[N] Whether to 0 = normalize / 1 = standardize / 2 = neither.	{**0**, 1, 2}
	-L < tolerance>	[N] The tolerance parameter.	{1.0E-1, 1.0E-2, **1.0E-3**, 1.0E-4, 1.0E-5}
	-P < epsilon>	[N] The epsilon for round-off error.	{1.0E-10, 1.0E-11, **1.0E-12**, 1.0E-14, 1.0E-15}
	-V < number>	[N] The number of folds for the internal cross-validation (default −1, use training data).	{**−1**, 5, 10}
	-W < seed>	[N] The random number seed.	{**1**, 3, 5, 7, 9}
	-M	[B] Fit calibration models to SVM outputs.	{true, **false**}
PART	-C < confidence>	[N] Set confidence threshold for pruning.	{0.1, **0.25**, 0.3, 0.4, 0.5}
	-M < number>	[N] Set minimum number of objects per leaf.	{1, **2**, 3, 4, 5}
	-R	[B] Use reduced error pruning.	{true, **false**}
	-B	[B] Use binary splits only.	{true, **false**}
	-U	[B] Generate unpruned decision list.	{true, **false**}
	-N < number>	[N] Set number of folds for reduced error pruning. One-fold is used as pruning set.	{1, 2, **3**, 4, 5}
	-Q < seed>	[N] Seed for random data shuffling.	{**1**, 3, 5, 7, 9}

Note that, [N] denotes the Numerical value, [B] denotes the Boolean value

5.6 RQ2: Does ChangeRanker outperform the baseline approaches in locating crash-inducing changes?

As aforementioned in Section 3, features selection can boost to the ranking performance. Therefore, we propose ChangeRanker that uses Rfs to select useful features to boosting crash-inducing change localization. Compared with ChangeLocator, ChangeRanker has the following advantages. First, it uses fewer features. Given its simplicity, a problem naturally arises: is ChangeRanker more effective than ChangeLocator in locating crash-inducing changes? If the answer is yes, this means that ChangeRanker is more practical for practitioners. Otherwise, we cannot justify the usefulness of rank-performance-based feature subset selection.

The current state-of-the-art supervised approach in locating crash-inducing changes is ChangeLocator proposed by Wu et al. Compared with Locus, it has a large performance improvement. In nature, ChangeRanker is an improved version of ChangeLocator by applying our Rfs feature selection. Therefore, it is necessary to use ChangeLocator as the baseline approach in our experiment. Besides, since Locus is the best unsupervised approach for locating crash-inducing changes, it is necessary to consider it as another baseline approach. To conduct a fair comparison, we use the same change candidates extracted for ChangeLocator instead of all previous changes as the input of Locus. Note that, we use the optimal configuration according to RQ1 for ChangeRanker.

5.7 RQ3: Is Rfs superior to other feature selection methods for ChangeRanker?

One of the contributions of this paper is that we propose a new feature selection method Rfs. In nature, ChangeRanker is the combination of ChangeLocator and Rfs. RQ2 evaluates whether Rfs can boost the performance of ChangeLocator. In the literature, various feature selection methods are available in the field of defect prediction. What if we replace Rfs with another feature selection method? This RQ aims to examine whether Rfs is superior to existing feature selection methods for boosting the performance of ChangeLocator. To this end, we systematically searched and investigated the feature selection methods in the literature. We found that, in 2017, on the basis of 18 defect data sets and 21 modeling techniques, Ghotra et al. (Ghotra et al. 2017) made a detailed experimental comparison and analysis of the effectiveness of 30 feature selection methods (12 wrapper-based subset selection techniques, six filter-based subset selection techniques, 11 filter-based feature ranking techniques, and a no feature selection configuration). They reported that: (1) of wrapper-based subset selection techniques, WS8 (i.e. Naïve Bayes with PRAUC optimization) performed the best; (2) of filter-based subset selection techniques, FS1 (i.e. correlation-based feature subset selection technique with

Table 5 The three popular feature selection baseline approaches

Family	Configuration	Weka Implementation	Abbr. in (Ghotra et al. 2017)
Wrapper-based	Classifier: Naive Bayes; Evaluation: PRAUC	weka.attributeSelection.WrapperSubsetEval	WS8
Filter-based	Type: Correlation-based; Search strategy: BestFirst	weka.attributeSelection.CfsSubsetEval	FS1
Ranking-based	Type: Classifier-based; Method: SVM	weka.attributeSelection.SVMAttributeEval	FR11

a BestFirst search) performed the best; and (3) of filter-based feature ranking techniques, FR11 (i.e. SVM-based technique) performed the best. In particular, similar findings are reported in (Xu et al. 2016). Therefore, we use WS8, FS1, and FR11 (the details shown in Table 5) as the representative feature selection methods to investigate RQ3.

5.8 RQ4: How ChangeRanker will perform when considering external factors from change candidates?

In the previous RQs, we have explored the optimal configuration and performance comparison of ChangeRanker. There are two external factors: the forms of change candidates and the size of change candidates set. The former can affect the prediction result, while the latter can affect the evaluation of an approach.

(1) **The forms of change candidates**

As mentioned in Section 2.4, there are three forms of candidate changes: Form1, Form2, and Form3. Form2 is a subset of Form1 by leveraging control flow analysis to exclude irrelevant changes, while Form3 is a subset of Form2 by leveraging backward slicing to further exclude irrelevant changes. In (Wu et al. 2018), Wu et al. reported that ChangeLocator performed the best on the candidate changes of Form3. Similarly, we want to investigate the effect on performance of ChangeRanker with different forms of change candidates and hence we can give a more appropriate recommendation for the use of ChangeRanker.

(2) **The size of change candidate set**

In the previous experiments, when computing Recall@k, we do not take into account the influence of change set size (i.e. the number of candidate changes for a bucket). For a given crash bucket that has an oracle, if the corresponding change set has a size less than or equal to k, the crash-inducing change must be found by examining the top k changes. Consequently, the ability of an approach in locating crash-inducing changes will be overestimated. Intuitively, such crash buckets are inappropriate to be used for computing Recall@k, as any two approaches (such as ChangeRanker and ChangeLocator) will have the same score on them. In other words, such crash buckets may have a confounding effect on when comparing the ranking effectiveness between two localization approaches. In order to obtain an objective evaluation on the ability of an approach to rank crash-inducing changes, we should exclude such buckets before computing Recall@k. We want to investigate the effect of the size of change candidate set on the effectiveness of ChangeRanker.

6 Experimental results

In this section, we report in detail the experimental results on the three research questions under study.[4] This will help determine the usefulness of ChangeRanker in practice.

[4] The data sets and scripts can be download via https://github.com/Naplues/ChangeRanker.

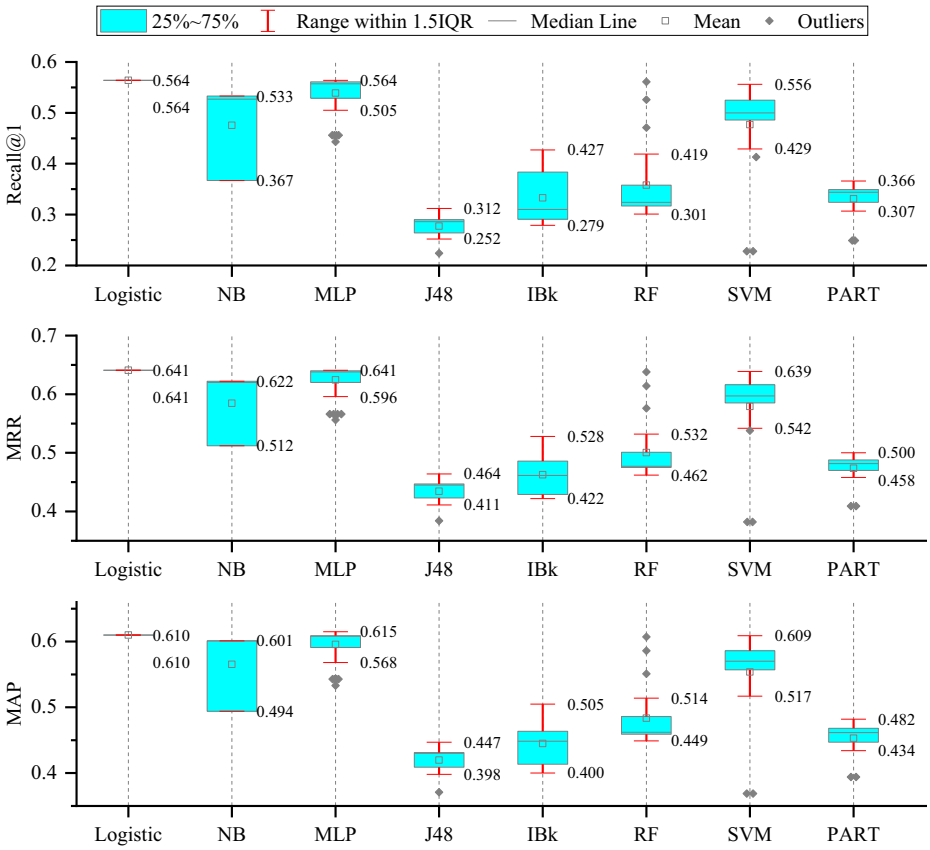


Fig. 5 The performance distributions (in terms of Recall@1, MRR, and MAP) of ChangeRanker under each modeling technique with different parameter settings in NetBeans

6.1 RQ1: How do the configuration factors affect the effectiveness of ChangeRanker?

(1) The influence of modeling techniques

Figure 5 reports the performance distributions (in terms of Recall@1, MRR, and MAP) of ChangeRanker under each modeling technique with different parameter settings in NetBeans. For the ease of observation, we annotate the maximum and minimum result for each classifier. It can be seen, there are differences among these classifiers when they are used to rank crash-inducing changes. First, we compare different classifiers in terms of their optimal performance. As shown in Fig.5, Logistic, MLP, and SVM perform better than others, which can achieve the Recall@1 values of 0.564, 0.564 and 0.556 respectively. Meanwhile, J48 achieve the worst performance values regardless of whether Recall@1 (0.312), MRR (0.464), or MAP (0.447) is considered. Second, we consider the sensitivity of each classifier to different parameter settings, which can measure the stability of a classifier. We find that, Logistic, MLP, J48, and PART have low sensitivity to different parameters (i.e., a narrow variance under different parameters). In particular, Logistic is insensitive to parameters, which achieve the same performance values when different parameter settings are used. Also, we can find that NB is

Table 6 The feature subset selected under different feature selection settings

Operator	"×"			"+"		
Test Set Version	Recall@1	MRR	MAP	Recall@1	MRR	MAP
V6.7	4 8 9	4 8 9	4 8 9	4 9	4 8 9	4 8 9
V6.8	4 8 9	4 8 9	4 8 9	4 8 9	4 8 9	4 8 9
V6.9	4 8 9	4 8 9	4 8 9	4 8 9	4 8 9	4 8 9
V7.0	4 8 9	4 8 9	4 8 9	4 8 9	4 8 9	4 8 9
V7.1	4 8 9	4 8 9	4 8 9	4 8 9	4 8 9	4 8 9
V7.2	4 8 9	4 8 9	4 9	4 8 9	4 8 9	4 8 9

Feature mapping: 0: NAF; 1: RLOCC; 2: RLOAC; 3: RLODC; 4: IADCP; 5: ITDCR; 6: RF; 7: IBF; 8: CC; 9: IADCL

sensitive to parameters. Third, we compare the complexity among different classifiers, which can be measured by the number of parameters. The fewer the number of parameters, the simpler the model, the easier to control and to use. Logistic and NB have lowest number (i.e., 2) of parameters while there are 14 parameters needed to be tuned for MLP.

In summary, Logistic is the appropriate classifier because of the high-performance values, low sensitivity to parameter settings, and low number of parameters compared with the other classifiers. In addition, MLP classifier also achieves a good performance while it has a large number of parameters need to be tuned. Meanwhile, the performance values of NB vary greatly when different parameters are used. Therefore, we recommend selecting logistic regression as the modeling technique for crash-inducing change localization.

(2) **The influence of feature selection settings**

Table 6 reports the feature set selected under different feature selection settings. It is clear that the entire same feature subset is obtained for the majority versions (i.e. v6.8-v7.1). The two exceptions are in setting (v7.2, MAP, "×") and in setting (v6.7, Recall@1, "+"), which select IADCP and IADCL as the superior features. According to the results, we think that we can replace MRR with the other two performance metrics in locating crash-inducing changes. What is more, the results in our experiment show that feature subset (IADCP, IADCL, and CC) is a good change-level features for the NetBeans project. The core observation is that: since the selected feature subsets are the same under most settings, the feature selection settings have little influence on the localization result of ChangeRanker.

(3) **The influence of maximum depth of feature combination tree**

Figure 6 reports the comparison result in terms of Recall@1, MRR, and MAP under different maximum depths. From the line chart, we find the same trend for ChangeRanker under different performance indicators. Since Recall@1, MRR, and MAP exhibit the same trend, in the following, we only use Recall@1 to analyze the trend how ChangeRanker varies with the maximum depth of feature combination tree. From Fig. 6(a), we can see that, as the maximum depth of feature combination tree increases from 1 to 3, Recall@1 improves gradually. ChangeRanker achieves the best Recall@1 when the maximum depth is equal to 3. However, Recall@1 begins to decrease when the maximum depth is larger than default value (i.e., 3). This indicates that uninformative features are introduced into ChangeRanker.

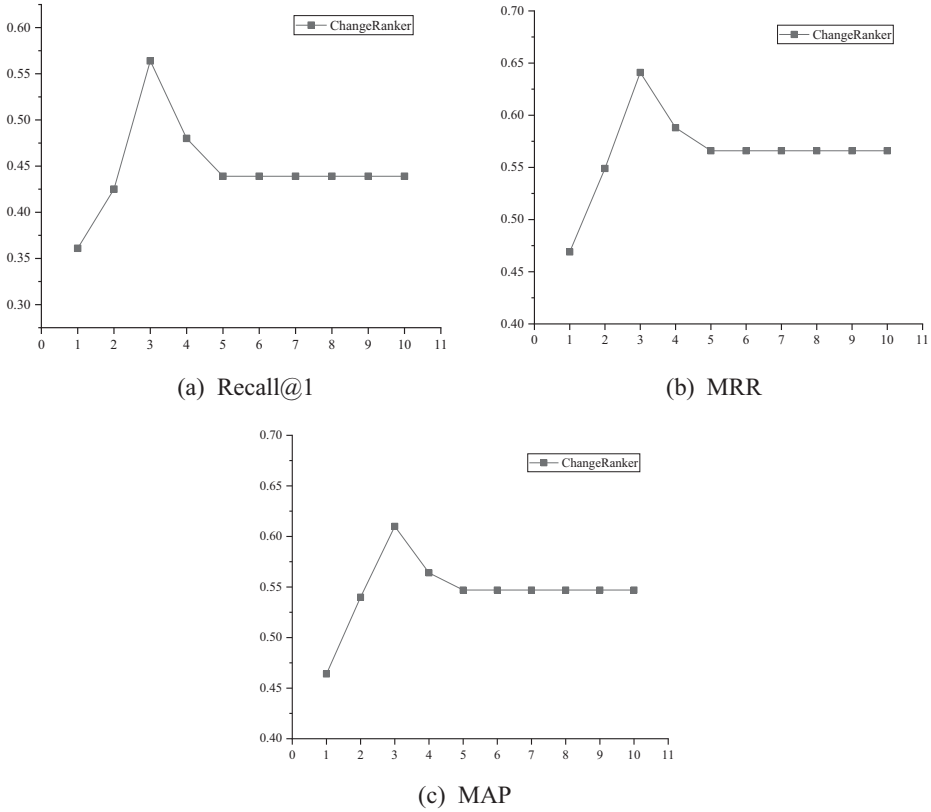


Fig. 6 The performance trend with different maximum feature combination number (a) Recall@1 (b) MRR (c) MAP

Note that, Recall@1 does not change when the maximum depth is larger than 5. This means that a larger feature subset (the number of features is larger than 5) cannot improve the performance of ChangeRanker. According to the trend, we can know that there are 5/10 features are useful for locating crash-inducing changes and 3 features are very prominent

Table 7 The feature subsets selected by applying different approaches

Maximum number	v6.7	v6.8	v6.9	v7.0	v7.1	v7.2
1	4	4	4	4	4	4
2	4 8	2 8	4 8	4 8	4 8	4 8
3	4 8 9	4 8 9	4 8 9	4 8 9	4 8 9	4 8 9
4	2 4 5 8	4 6 8 9	2 4 8 9	2 4 8 9	2 4 8 9	2 4 8 9
5	2 4 5 6 8	0 4 5 8 9	2 4 8 9	2 4 8 9	2 4 8 9	2 4 8 9
6	2 4 5 6 8	0 4 5 8 9	2 4 8 9	2 4 8 9	2 4 8 9	2 4 8 9
7	2 4 5 6 8	0 4 5 8 9	2 4 8 9	2 4 8 9	2 4 8 9	2 4 8 9
8	2 4 5 6 8	0 4 5 8 9	2 4 8 9	2 4 8 9	2 4 8 9	2 4 8 9
9	2 4 5 6 8	0 4 5 8 9	2 4 8 9	2 4 8 9	2 4 8 9	2 4 8 9
10	2 4 5 6 8	0 4 5 8 9	2 4 8 9	2 4 8 9	2 4 8 9	2 4 8 9

Feature mapping: 0: NAF; 1: RLOCC; 2: RLOAC; 3: RLODC; 4: IADCP; 5: ITDCR; 6: RF; 7: IBF; 8: CC; 9: IADCL

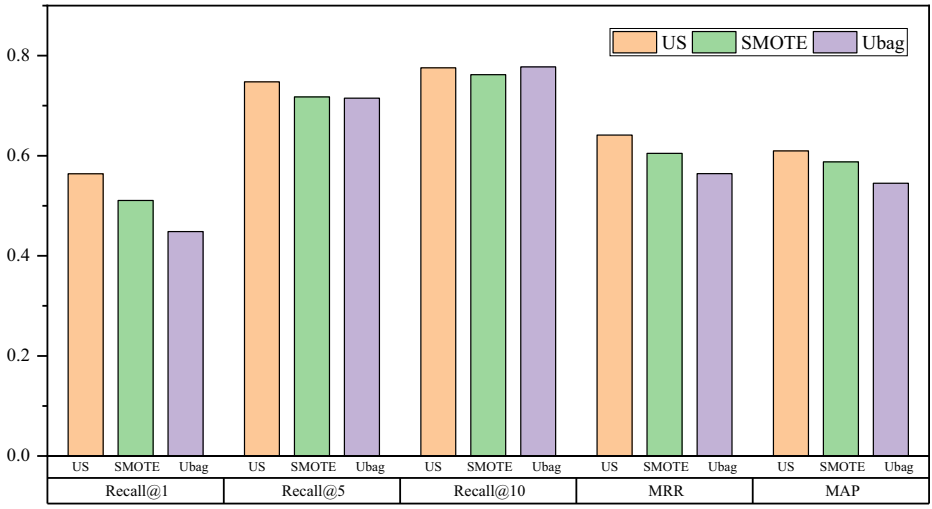


Fig. 7 The performance comparison with different class-imbalance solutions

between them. This indicates that our ranking-performance-based feature selection is essential for improving the performance of locating crash-inducing changes.

Table 7 reports the feature subsets used by different approaches. For ChangeRanker, we can see that feature 4 (i.e., IADCP) is selected when the maximum depth is set to 1. This indicates that IADCP is the most important feature for locating crash-inducing changes. In addition, we get the best feature subset {4, 8, 9} (i.e., IADCP, CC, and IADCL) when the maximum depth is set to 3, which includes the useful features (i.e. IADCP and CC) we find in the motivational example in Section 3. Interestingly, we can see that, when the maximum depth varies from 5 to 10, the subset selected by Rfs is stable, regardless of which NetBeans version is considered. In particular, Rfs selects the same feature subset {2, 4, 8, 9} (i.e., RLOAC, IADCP, CC, and IADCL) for NetBeans 6.9, 7.0, 7.1, and 7.2.

Combining the results from Fig. 6 and Table 7, we conclude that the maximum depth of feature combination tree may have an influence on the effectiveness of ChangeRanker.

(4) **The influence of class-imbalance strategies**

Figure 7 reports the performance comparison with different class-imbalance strategies. For each indicator, we calculate the performance scores according to the candidate from three forms for each strategy. From Fig. 7, we can see that, overall, both SMOTE and UBag lead to an inferior performance than US. This indicates that US is the proper solution to provide the balanced training data for locating crash-inducing changes.

According to above results, we obtain the optimal setting of configuration factors of ChangeRanker. That is, the best modeling technique is Logistic, the best settings for feature selection is (MRR, "×"), the most suitable maximum depth of feature combination tree is 3, and the optimal class-imbalanced strategies is US. In the following experiments, we apply the optimal setting to get the result of ChangeRanker.

Table 8 The comparison of ranking effectiveness between ChangeRanker and ChangeLocator in NetBeans

Version	Approach	Recall@1	Recall@5	Recall@10	MRR	MAP
v6.7	Locus	0.098	0.361	0.590	0.233	0.222
	ChangeLocator	0.230	0.672	**0.754**	0.415	0.401
	ChangeRanker	**0.443**	**0.738**	**0.754**	**0.565**	**0.554**
v6.8	Locus	0.154	0.442	0.538	0.265	0.253
	ChangeLocator	0.365	**0.654**	**0.750**	0.506	0.501
	ChangeRanker	**0.558**	0.635	0.712	**0.604**	**0.573**
v6.9	Locus	0.075	0.300	0.500	0.182	0.168
	ChangeLocator	0.375	**0.700**	**0.725**	0.504	0.495
	ChangeRanker	**0.500**	0.675	0.675	**0.572**	**0.536**
v7.0	Locus	0.079	0.316	0.553	0.203	0.197
	ChangeLocator	0.447	0.658	**0.789**	0.554	0.526
	ChangeRanker	**0.605**	**0.763**	**0.789**	**0.656**	**0.618**
v7.1	Locus	0.049	0.341	0.585	0.205	0.201
	ChangeLocator	0.537	0.805	**0.878**	0.641	0.609
	ChangeRanker	**0.610**	**0.829**	**0.878**	**0.708**	**0.677**
v7.2	Locus	0.103	0.333	0.487	0.218	0.218
	ChangeLocator	0.538	0.821	**0.846**	0.658	0.642
	ChangeRanker	**0.667**	**0.846**	**0.846**	**0.743**	**0.700**
Avg.	Locus	0.093	0.349	0.542	0.218	0.210
	ChangeLocator	0.415	0.718	**0.790**	0.546	0.529
	ChangeRanker	**0.564**	**0.748**	0.776	**0.641**	**0.610**

6.2 RQ2: Does ChangeRanker outperform the baseline approaches in locating crash-inducing changes?

Table 8 summarizes the ranking performance for ChangeRanker and ChangeLocator in NetBeans. In this table, we bold the best score for the same indicator in each version. In terms of the average Recall@1 and Recall@5, ChangeRanker can achieve 0.564 and 0.748, which are better than that of ChangeLocator. In terms of the average Recall@10, ChangeRanker is slightly lower than ChangeLocator. When looking at the score of each version, we observe that ChangeRanker achieves the best performance in all the six versions in terms of Recall@1 and in most versions (4/6) in terms of Recall@5 and Recall@10. According to these results, we conclude that ChangeRanker can find more crash-inducing changes when reviewing top 1 change or top 5 changes. Furthermore, on average, ChangeRanker exhibits an improvement of 17.40% in MRR and an improvement of 15.31% in MAP over ChangeLocator. In particular, ChangeRanker outperforms

Table 9 The statistical test results between ChangeRanker and ChangeLocator in NetBeans

Baselines	Statistical test	Recall@1	Recall@5	Recall@10	MRR	MAP
Locus	p value	**0.042**	**0.042**	**0.042**	**0.042**	**0.042**
	cliff's delta | δ|	1.000	1.000	1.000	1.000	1.000
	effect size	Large	Large	Large	Large	Large
ChangeLocator	p value	**0.042**	0.253	0.398	**0.042**	**0.042**
	cliff's delta | δ|	0.722	0.278	0.111	0.611	0.611
	effect size	Large	Small	Negligible	Large	Large

A bold font denotes the Benjamini-Hochberg corrected Wilcox signed-rank test p value <0.05

Table 10 The comparison of ranking effectiveness between Rfs and other feature selection methods in NetBeans

Versions	FS Approach	Recall@1	Recall@5	Recall@10	MRR	MAP
v6.7	WS8	0.344	0.738	0.754	0.507	0.500
	FS1	0.279	0.721	0.738	0.456	0.448
	FR11	0.361	0.705	0.754	0.498	0.489
	Rfs	**0.443**	**0.738**	**0.754**	**0.565**	**0.554**
v6.8	WS8	0.346	**0.673**	**0.750**	0.501	0.488
	FS1	0.250	0.615	0.712	0.424	0.423
	FR11	0.308	0.615	0.731	0.445	0.435
	Rfs	**0.558**	0.635	0.712	**0.604**	**0.573**
v6.9	WS8	0.350	0.675	**0.700**	0.459	0.443
	FS1	0.475	**0.700**	**0.700**	0.552	**0.536**
	FR11	0.350	0.650	**0.700**	0.471	0.454
	Rfs	**0.500**	0.675	0.675	**0.572**	**0.536**
v7.0	WS8	0.237	0.684	**0.789**	0.423	0.410
	FS1	0.342	0.684	0.763	0.491	0.454
	FR11	0.289	0.711	**0.789**	0.443	0.430
	Rfs	**0.605**	**0.763**	**0.789**	**0.656**	**0.618**
v7.1	WS8	0.390	0.780	0.854	0.569	0.539
	FS1	0.268	0.683	0.805	0.452	0.434
	FR11	0.293	0.683	**0.878**	0.465	0.448
	Rfs	**0.610**	**0.829**	**0.878**	**0.708**	**0.677**
v7.2	WS8	0.308	0.769	**0.846**	0.496	0.482
	FS1	0.333	0.769	0.821	0.514	0.492
	FR11	0.308	0.744	**0.846**	0.469	0.443
	Rfs	**0.667**	**0.846**	**0.846**	**0.743**	**0.700**
Avg.	WS8	0.329	0.720	0.782	0.493	0.477
	FS1	0.325	0.695	0.757	0.482	0.465
	FR11	0.318	0.685	**0.783**	0.465	0.450
	Rfs	**0.564**	**0.748**	0.776	**0.641**	**0.610**

ChangeLocator over all of the six versions, regardless of whether MRR or MAP is considered. In summary, ChangeRanker has a considerable improvement on ChangeLocator in NetBeans.

Table 9 reports the statistical test results between ChangeRanker and ChangeLocator in NetBeans. As can be seen, according to the BH-corrected p value, ChangeRanker is significantly better than the baseline approaches at the significance level of 0.05, regardless of whether Recall@1, MRR, or MAP is considered. When looking at the effect size, we can find that the difference between ChangeRanker and ChangeLocator are large ($|\delta| \geq 0.474$), regardless of whether Recall@1, MRR, or MAP is considered.

6.3 RQ3: Is Rfs superior to other feature selection methods for ChangeRanker?

Table 10 reports the average ranking performance comparison between Rfs and other feature selection methods in NetBeans. Bold scores in each version denote the best performance among different methods. As can be seen, on average, Rfs perform better than the feature selection baselines in terms of Recall@1, MRR and MAP. In particular, on NetBeans 6.7, 7.0, 7.1 and 7.2, ChangeRanker achieves the best performance on all five indicators. Of the three baseline methods, WS8 performs better than the others on average. However, Rfs has an improvement of 71.4%, 30%, and 27.9% over WS8 in terms of Recall@1, MRR, and MAP, respectively. The results in NetBeans show that Rfs is more advantageous for crash-inducing change localization.

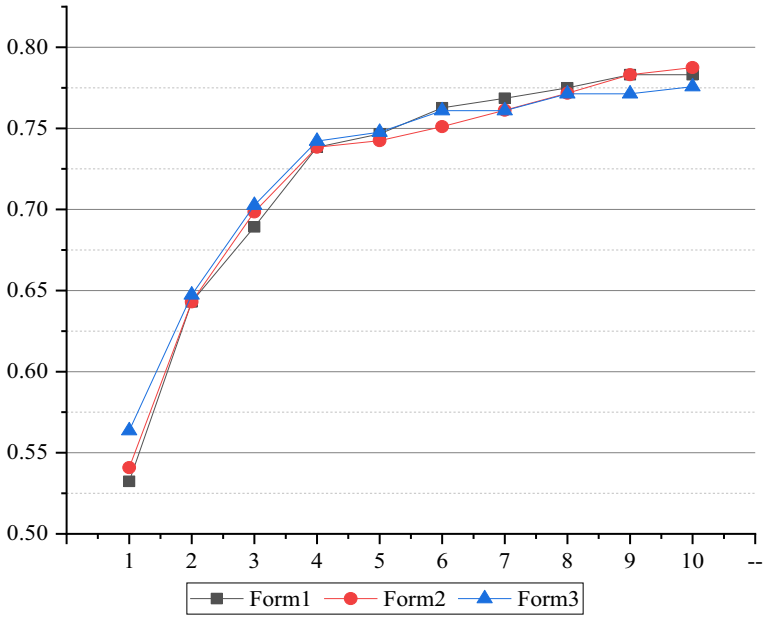


Fig. 8 Recall@K of ChangeRanker on different forms of candidate changes in NetBeans

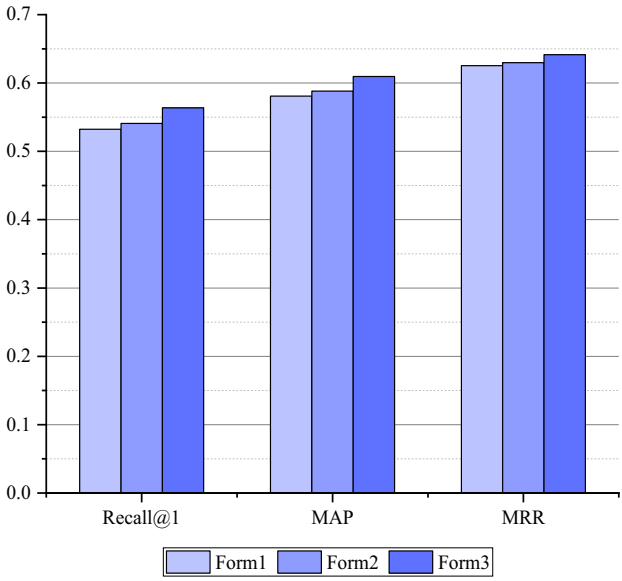


Fig. 9 The ranking performance of ChangeRanker on different forms of candidate changes in NetBeans

6.4 RQ4: How ChangeRanker will perform when considering external factors from change candidates?

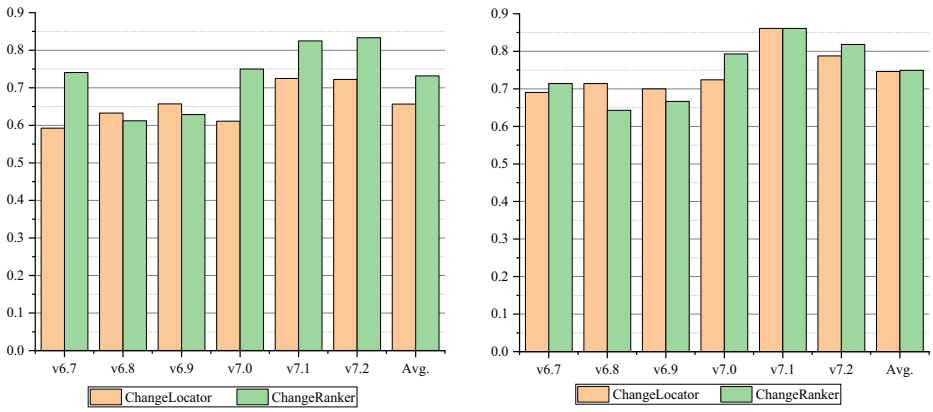
(1) **The forms of change candidates**

How do different forms of candidate changes affect the effectiveness of ChangeRanker? Figure 8 shows for ChangeRanker the Recall@K when K varies between 1 and 10. As can be seen, the candidate changes of Form 3 leads to a considerably higher Recall@1. However, when $K \geq 2$, ChangeRanker has a similar Recall@K under different forms of candidate changes. Figure 9 shows for ChangeRanker the average Recall@1, MRR, and MAP over the six NetBeans versions under different forms of candidate changes. We can observe that Form3 performs better than Form2, which performs better than Form 1. Therefore, it appears that excluding irrelevant changes helps improve the ranking effectiveness. Overall, the above results suggest that we should use ChangeRanker combined with the changes of Form3 in practice.

(2) **The size of change candidates set**

As shown in Table 3, the minimum change set size is 2 over the six NetBeans versions. In particular, of the 208 crash buckets having an oracle, there are 19 (51) crash buckets whose change set size is less or equal to 5 (10). In Section 6, when computing Recall@k ($k = 5$, 10), such crash buckets were not excluded. Not that, the number of low change set size buckets is less (5 for k = 5 and 10 for k = 10). In the following, we use the filtered crash buckets to re-compute Recall@k in NetBeans: (1) before computing Recall@5, exclude the 19 crash buckets whose change set size is less or equal to 5; and (2) before computing Recall@10, exclude the 51 crash buckets whose change set size is less or equal to 10.

Figure 10 compares the ranking effectiveness between ChangeRanker and ChangeLocator based on the filtered crash buckets in NetBeans. As can be seen, in terms of Recall@5, ChangeRanker has a higher average performance than ChangeLocator. In terms of Recall@10,



(a) Recall@5 (b) Recall@10

Fig. 10 The comparison of ranking effectiveness in NetBeans: ChangeRanker vs. ChangeLocator

ChangeRanker has a similar average performance as ChangeLocator. The above results indicate that, after excluding those crash buckets with small change sets, our conclusion on the superiority of ChangeRanker remains no change.

(For Recall@k, the crash buckets whose change set size is less than or equal to k have been excluded) (a) Recall@5 (b) Recall@10.

7 Examination of generalization

The experimental results in Section 6 show that our proposed approach ChangeRanker has a good performance in locating crash-inducing changes in NetBeans. To examine the generalization capability of our approach, we use additional datasets to conduct more experiments.

7.1 Defect-inducing change localization

ChangeRanker is originally designed to boost ChangeLocator in locating crash-inducing changes. In Section 6, our experimental results are only based on the NetBeans project. It is unclear whether our findings can be generalized to the other projects. Ideally, we should use crash-inducing change data sets from diverse projects to evaluate the effectiveness of ChangeRanker. However, as stated in (Wu et al. 2018), it is difficult to collect accurate crash data from other projects such as Firefox and Eclipse. This is the reason why Wu et al. evaluate the effectiveness of ChangeLocator using only the Netbeans project. However, this is a threat to the external validity of ChangeLocator, which is also applicable to ChangeRanker. In order to alleviate this threat, we adapt ChangeLocator and ChangeRanker to localize defect-inducing changes. As we know, software crash is a specific type of software defects. In the view of this fact, the approaches for locating crash-inducing changes can be adapted to the problem of defect-inducing change localization.

7.2 Data pre-processing

Wen et al. collected the defect-introducing changes for each defect in six open-source projects (Wen et al. 2016). According to their study, the datasets can be used to evaluate the effectiveness of a defect-inducing change localization approach. In our study, we leverage their data sets to evaluate the generalization capability of ChangeRanker. To this end, we extract 10 change-level features from the datasets provided by Wen et al. Their datasets contain bug reports and the corresponding defect-inducing changes information for each bug. Note that, a bug report is somewhat different from a crash report since there may have no stack trace in the bug report. ChangeRanker cannot extract features for the bug reports without any program stack trace. Because change-level features cannot be extracted directly, we take the following pre-processing steps when extracting features.

1) assigning bucket To be consistent with NetBeans project, we take a bug report as a bucket and use the bug ID to represent the bucket.

2) extracting crash stack A bug report usually is composed of natural language description and crash stack traces. We read each bug report and extract the program stack information

Table 11 The detail information on additional projects

project	#files	KLOC	#Revs	#bucket	#training	#testing (#oracles /#c <=5/=10)
AspectJ	7037	409	7760	39	20	19(8/2/3)
JDT	7104	1700	22,469	24	9	15(15/0/0)
Tomcat	2415	487	20,792	38	18	20(18/1/2)
Total	16,556	2596	51,021	101	47	54(41/3/5)

manually as the crash stack. Note that, we discard the bug reports that do not contain any crash stack.

3) redefining CC feature Unlike NetBeans, the component to a bug is not specified explicitly in Wen et al.'s datasets so that we cannot extract the CC features defined by Wu et al. To solve the problem, we redefine the CC as follows. According to Wu et al. study (Wu et al. 2014), a bug is likely to appear in the source files occurred in the stack trace. The definition based on the heuristic that a change possibly introduces a bug if it modifies the files occurring in the crash stack trace. Let $F_{affected}(r)$ be source files affected by a revision r and $F_{stack}(B)$ be the files occurred in the stack traces for a bucket B. Then, we have:

$$CC(r,B) = \begin{cases} 1, F_{affected}(r) \cap F_{stack}(B) \neq \varnothing \\ 0, F_{affected}(r) \cap F_{stack}(B) = \varnothing \end{cases} \tag{7}$$

Table 11 reports the detail information on additional projects. In total, we use the three popular Java projects since their bug reports have enough stack traces used for our study. Note that, each project has only one version. Given this fact, we use time-wise cross-validation to evaluate the effectiveness of a defect-inducing localization approach. To this end, we first sort buckets in ascending order according to their bug fix time. Then, the change-level data of the top N buckets are combined as the training data, while the change-level data of the remaining buckets are combined as the test data. For each project, we use Green et al.'s suggestion (Wilson et al. 2007; Green 1991) to determine the value of N: the training data consists of at least $50 + 8\times m$ data points, where m is the number of features. In our context, we select a

Table 12 The comparison of ranking effectiveness between ChangeRanker and the baselines

Project	Approach	Recall@1	Recall@5	Recall@10	MRR	MAP
AspectJ	Locus	**0.105**	**0.158**	**0.158**	**0.118**	0.059
	ChangeLocator	0.000	0.053	**0.158**	0.044	0.044
	ChangeRanker	0.053	**0.158**	**0.158**	0.096	**0.096**
JDT	Locus	0.133	0.533	0.800	0.344	0.186
	ChangeLocator	0.200	**0.800**	**0.867**	0.445	0.370
	ChangeRanker	**0.467**	**0.800**	0.800	**0.578**	**0.529**
Tomcat	Locus	0.150	0.450	**0.700**	0.317	0.175
	ChangeLocator	0.250	0.600	**0.700**	0.405	0.401
	ChangeRanker	**0.450**	**0.650**	**0.700**	**0.555**	**0.526**
Avg.	Locus	0.129	0.380	0.553	0.260	0.140
	Improvement	+150.39%	+41.05%	0.00%	+57.69%	+174.29%
	ChangeLocator	0.150	0.484	**0.575**	0.298	0.272
	Improvement	+115.33%	+10.74%	−0.040%	+37.58%	+41.18%
	ChangeRanker	**0.323**	**0.536**	0.553	**0.410**	**0.384**

Table 13 The comparison of ranking effectiveness between Rfs and other feature selection methods

Project	FS Approach	Recall@1	Recall@5	Recall@10	MRR	MAP
AspectJ	WS8	0.000	**0.158**	**0.158**	0.061	0.061
	FS1	**0.053**	**0.158**	**0.158**	0.092	0.092
	FR11	0.000	0.053	**0.158**	0.043	0.043
	Rfs	**0.053**	**0.158**	**0.158**	**0.096**	**0.096**
JDT	WS8	0.400	0.667	0.733	0.525	0.483
	FS1	0.067	0.733	**0.867**	0.344	0.296
	FR11	0.333	0.733	0.733	0.491	0.462
	Rfs	**0.467**	**0.800**	0.800	**0.578**	**0.529**
Tomcat	WS8	0.200	0.600	**0.700**	0.380	0.357
	FS1	0.200	0.600	**0.700**	0.379	0.356
	FR11	0.200	0.550	**0.700**	0.340	0.316
	Rfs	**0.450**	**0.650**	**0.700**	**0.555**	**0.526**
Avg.	WS8	0.200	0.475	0.530	0.322	0.300
	Impr.	+61.67%	+12.84%	+4.21%	+27.23%	+27.75%
	FS1	0.107	0.497	**0.575**	0.272	0.248
	Impr.	+203.13%	+7.85%	−3.88%	+50.80%	+54.70%
	FR11	0.178	0.445	0.530	0.291	0.274
	Impr.	+81.99%	+20.36%	+4.21%	+40.62%	+40.19%
	Rfs	**0.323**	**0.536**	0.553	**0.410**	**0.384**

minimum N such that the corresponding buckets contain at least $50 + 8 \times 10 = 130$ changes. In this way, we determine $N = 20$ for AspectJ, $N = 9$ for JDT, and $N = 18$ for Tomcat.

7.3 Experimental results

Table 12 summarizes the ranking performance for ChangeRanker and baseline approaches on additional projects. As can be seen, ChangeRanker performs better than ChangeLocator in the majority of indicators (except for Recall@10). In terms of Recall@1, MRR, and MAP, ChangeRanker respectively achieves an improvement of 115.3%, 37.6%, and 41.2% over ChangeLocator on average. Compared with Locus, ChangeRanker achieves an improvement of 150.4%, 57.7%, and 174.3%, respectively. The results indicate that ChangeRanker ranks more defect-inducing changes on top than baseline approaches. The above results on additional projects clearly reveal that ChangeRanker performs better than ChangeLocator and Locus in locating defect-inducing changes. This indicates that ChangeRanker has a stronger ability to capture the relationship between features and defect-inducing changes. In summary, the superiority of ChangeRanker reflects its usefulness in locating defect-inducing changes. Consequently, for practitioners, it would be better to apply ChangeRanker to locate defect-inducing changes.

Table 13 reports the comparison of the average ranking performance between Rfs and three other feature selection technologies. As can be seen, in terms of Recall@1, Rfs achieves promising scores compared with all the other feature selection methods (the improvement is ranging from 61% to 203%). Considering MRR and MAP, Rfs can achieve the scores of 0.410 and 0.384, which has an improvement of 27.23% and 27.75% respectively. The results show that, Rfs is more competitive to selecting useful features when applying ChangeRanker.

Table 14 An example illustrating the effectiveness of ChangeRanker: The feature distribution of change candidates of bucket 146,027 (Form3) in NetBeans 6.7

Candidates	NAF	RLOCC	RLOAC	RLODC	*IADCP*	ITDCR	RF	IBF	*CC*	*IADCL*	isInducing
26847aebc78f	0.573	0.73	0.73	0.37	**1**	0.111	**1**	0.797	**1**	**1**	**TRUE**
870bdb7ff9c8	0.116	0.354	0.347	0.219	0.25	**0.143**	1	0.072	0	1	FALSE
81da823f2bd5	0.068	0.062	0.036	0.069	0.333	0.077	1	**1**	0	0.027	FALSE
f7b6c8832a63	0.268	0.338	0.299	0.342	1	0.059	1	**1**	0	0.1	FALSE
ab06cad09216	0.153	0.428	0.421	0.282	0.333	**0.5**	1	**1**	0	1	FALSE
d3cf65212a44	0	0.082	0	0.138	1	0.091	1	0.797	0	1	FALSE
493703d05279	**0.695**	**0.858**	**0.858**	0.179	1	0.05	1	0.304	0	1	FALSE
9f5d40ff4857	0.116	0.325	0.153	**0.416**	0.5	**0.125**	1	**1**	0	0.5	FALSE
060cd4367ad5	0.068	0.422	0.408	0.351	0.5	**1**	1	**1**	0	1	FALSE
46d53e788b9a	0.068	0.32	0.314	0.179	0.5	0.067	1	**1**	0	0.091	FALSE
8870deb3cd89	**0.736**	**0.734**	0.69	**0.788**	0.2	**0.167**	1	0.072	0	0.25	FALSE
28b0f56319e6	0.116	0.301	0.292	0.179	0.333	0.071	1	**1**	0	0.167	FALSE
0a76d1857ab0	0.312	0.615	0.564	**0.668**	1	**0.25**	1	**1**	1	0.059	FALSE
a6b6e3cf36a4	**0.901**	**1**	**1**	0.069	0.25	0.1	1	0	0	1	FALSE
b70d80b11b89	0.116	0.373	0.339	**0.372**	0.5	0.063	1	**1**	0	0.143	FALSE
f23762fedb10	0	0.223	0.192	0.207	1	0.056	1	**1**	0	0.1	FALSE
37ed1a51c9e3	0	0.24	0.168	0.282	0.5	**0.333**	1	**1**	0	0.167	FALSE
7cd6624f75c6	0.068	0.24	0.234	0.109	0.333	**0.2**	1	**1**	0	0.5	FALSE
94f45bbfb018	0.251	0.39	0.336	**0.425**	1	0.053	1	**1**	0	0.167	FALSE
9808bc15dd16	0	0.302	0.259	0.312	0.5	0.083	1	**1**	0	0.2	FALSE
#bold candidates	3	3	2	5	0	8	0	14	0	0	

Table 15 An example illustrating the effectiveness of ChangeRanker: The ranking results output by ChangeLocator and ChangeRanker for bucket 146,027 in NetBeans 6.7

Rank	ChangeLocator			ChangeRanker		
	Candidates	Score	IsInducing	Candidates	Score	IsInducing
1	f7b6c8832a63	1	FALSE	**26847aebc78f**	**0.995146**	**TRUE**
2	f23762fedb10	1	FALSE	0a76d1857ab0	0.789321	FALSE
3	d3cf65212a44	1	FALSE	d3cf65212a44	5.28E-07	FALSE
4	94f45bbfb018	1	FALSE	493703d05279	5.28E-07	FALSE
5	870bdb7ff9c8	1	FALSE	060cd4367ad5	5.99E-08	FALSE
6	493703d05279	1	FALSE	ab06cad09216	2.90E-08	FALSE
7	**26847aebc78f**	**1**	**TRUE**	a6b6e3cf36a4	2.02E-08	FALSE
8	0a76d1857ab0	1	FALSE	870bdb7ff9c8	2.02E-08	FALSE
9	a6b6e3cf36a4	1	FALSE	94f45bbfb018	1.53E-08	FALSE
10	8870deb3cd89	2.07E-07	FALSE	f7b6c8832a63	1.15E-08	FALSE
11	9f5d40ff4857	5.72E-28	FALSE	f23762fedb10	1.15E-08	FALSE
12	37ed1a51c9e3	1.14E-45	FALSE	9f5d40ff4857	7.14E-09	FALSE
13	060cd4367ad5	4.51E-46	FALSE	7cd6624f75c6	3.46E-09	FALSE
14	9808bc15dd16	2.44E-47	FALSE	9808bc15dd16	1.99E-09	FALSE
15	b70d80b11b89	1.59E-57	FALSE	37ed1a51c9e3	1.73E-09	FALSE
16	46d53e788b9a	1.36E-58	FALSE	b70d80b11b89	1.56E-09	FALSE
17	81da823f2bd5	6.16E-59	FALSE	46d53e788b9a	1.25E-09	FALSE
18	7cd6624f75c6	1.39E-65	FALSE	28b0f56319e6	8.38E-10	FALSE
19	ab06cad09216	2.49E-76	FALSE	8870deb3cd89	6.69E-10	FALSE
20	28b0f56319e6	9.38E-84	FALSE	81da823f2bd5	4.63E-10	FALSE

In summary, the above results indicate that our proposed extension to ChangeLocator has a good generalization capability.

8 Discussion

In this section, we first use examples to understand why ChangeRanker works better and when ChangeRanker does not work well. Then, we discuss the implications of our results for researchers and practitioners.

8.1 Why ChangeRanker works well?

Table 14 and Table 15 present an example for illustrating the effectiveness of ChangeRanker. Table 14 reports the feature distribution of change candidates of bucket 146,027 in NetBeans 6.7. The bold values mark the change candidates that have a larger value than crash-inducing change (i.e., change 26847aebc78f) in one feature. The feature with many bold change candidates may indicate that it is not significant for locating crash-inducing change(s) since these candidates would be ranked top when considering the feature. In other words, these changes have a greater degree of suspicion when considering this feature according to the definition. The last row summaries the number of bold candidates for each feature. The more bold candidates a feature has, the less important it is. It can be seen that, the feature ITDCR and IBF have more bold change candidates. In particular, the proportion of bold candidates of IBF are more than half (70%), which indicates the feature is not important. These features can disturb the prediction accuracy of ChangeLocator largely. The columns with italic feature

Table 16 An example illustrating the misclassification of ChangeRanker: The feature distribution of change candidates of bucket 171,622 (Form3) in NetBeans 7.0

Candidates	NAF	RLOCC	RLOAC	RLODC	*IADCP*	ITDCR	RF	IBF	*CC*	*IADCL*	isInducing
ea19d529cada	0.131	0.523	0.523	0.068	0.25	**1**	**1**	**1**	0	0.2	**TRUE**
870bdb7ff9c8	0.099	0.354	0.347	**0.217**	0.143	0.1	1	0.077	0	**1**	FALSE
313ca82fe335	**1**	**0.938**	**0.896**	**1**	0.029	0.111	0	0.11	**1**	**1**	FALSE
b0be637aefcc	**0.216**	0.391	0.329	**0.429**	0.25	0.25	1	1	0	0.167	FALSE
7ca019a7e8e1	**0.245**	0.388	0.322	**0.431**	**0.333**	0.125	1	1	**1**	**1**	FALSE
a6b6e3cf36a4	**0.775**	**1**	**1**	0.068	0.172	0.091	1	0	0	**1**	FALSE
2bc794b05dce	0.058	0.395	0.391	**0.217**	0.125	0.143	1	0.077	0	**0.5**	FALSE
fedb8654e286	0.099	0.342	0.281	**0.375**	**0.278**	0.2	1	1	**1**	**1**	FALSE
86fd70be02a5	0.131	0.062	0.062	0	0.167	0.5	1	1	**1**	**0.5**	FALSE
d208fe7728d5	**0.323**	**0.558**	**0.558**	0	0.25	0.167	1	1	0	**1**	FALSE
95da06f756f9	0.058	0.298	0.292	**0.137**	0.143	0.333	1	0.151	0	**0.333**	FALSE
#bold candidates	5	3	3	7	2	0	0	0	4	9	

names show the features (selected by Rfs) used in ChangeRanker. As can be seen, there are no bold candidates in the selected features. Therefore, ChangeRanker can work and get a better performance.

In Table 15, we use bold font to mark the position of the crash-inducing change in the ranked list for ChangeLocator and ChangeRanker. As can be seen, the suspicious score of the crash-inducing change is equal to 8 other candidate changes in the list generated by ChangeLocator. Although the suspicious score of crash-inducing change is the biggest, the model cannot decide which one should be ranked in the first place. In this example, it is ranked 7th. Meanwhile, ChangeRanker put the crash-inducing change to the first place in the list. By excluding these unimportant features, the performance of model can be improved greatly.

8.2 When ChangeRanker does not work well?

Previous discussion shows the situation that ChangeRanker performs well for locating crash-inducing changes. However, ChangeRanker cannot output an excellent ranking list for all buckets. There are still very few parts of buckets that ChangeRanker cannot work well. Table 16 and Table 17 report another bucket (177162) example to illustrate the misclassification of ChangeRanker. As can be seen, there are many features (7/10) with the bold change candidates (i.e., change candidates that have a larger value than the crash-inducing change). In Section 9.1, we can see that such features could affect negatively for the ranking result and hence they should be regarded as insignificant features. However, it can be seen that, there are also many bold change candidates for the features (selected by Rfs) used by ChangeRanker. In particular, for the values of IADCL, only one change candidate (i.e., b0be637aefcc) has a smaller score than the crash-inducing change. This means that many changes would be ranked higher than the crash-inducing change when considering IADCL. According to the ranking results shown in Table 17, the crash-inducing change is ranked 10th. We can see that, both ChangeLocator and ChangeRanker have poor ranking results. The reason for this is that the features used by both approaches are insignificant for bucket 171,622. For these buckets, ChangeRanker cannot work well.

Table 17 An example illustrating the misclassification of ChangeRanker: ranking results by ChangeLocator and ChangeRanker for bucket 171,622 in NetBeans 7.0

Rank	ChangeLocator			ChangeRanker		
	Candidates	Score	IsInducing	Candidates	Score	IsInducing
1	fedb8654e286	0.899	FALSE	7ca019a7e8e1	0.895	FALSE
2	7ca019a7e8e1	0.892	FALSE	fedb8654e286	0.887	FALSE
3	313ca82fe335	0.745	FALSE	313ca82fe335	0.847	FALSE
4	86fd70be02a5	0.530	FALSE	86fd70be02a5	0.657	FALSE
5	d208fe7728d5	0.183	FALSE	d208fe7728d5	0.108	FALSE
6	a6b6e3cf36a4	0.074	FALSE	a6b6e3cf36a4	0.098	FALSE
7	b0be637aefcc	0.036	FALSE	870bdb7ff9c8	0.094	FALSE
8	**ea19d529cada**	**0.034**	**TRUE**	2bc794b05dce	0.028	FALSE
9	870bdb7ff9c8	0.021	FALSE	95da06f756f9	0.019	FALSE
10	2bc794b05dce	0.008	FALSE	**ea19d529cada**	**0.016**	**TRUE**
11	95da06f756f9	0.007	FALSE	b0be637aefcc	0.015	FALSE

8.3 What are the implications of our results?

Our study has two implications for researchers: 1) we propose an effective approach ChangeRanker to boost the effectiveness of locating crash-inducing changes. The approach can rank crash-inducing changes higher than the baselines in most cases. Because of its outstanding performance, we believe that our work provides an opportunity for further study to gain more insights into localizing crash-inducing changes; 2) our proposed Rfs provides a simple yet effectiveness feature selection method. As shown in our experiment, in the ranking scenario, Rfs has a better performance than the best-performing feature selection methods in the literature. As a result, we suggest that researchers should consider using Rfs to select the informative features in the future work.

9 Threats to validity

We consider the most important threats to construct, internal, and external validity of our study. Construct validity is the degree to which the dependent and independent variables accurately measure the concept they purport to measure. Internal validity is the degree to which conclusions can be drawn about the causal effect of independent variables on the dependent variables. External validity is the degree to which the results of the research can be generalized to the population under study and other research settings.

9.1 Construct validity

The dependent variable used in this study is a binary variable indicating whether a candidate change is crash-inducing. Our study used the same datasets as used in Wu et al.'s study. According to (Wu et al. 2018), the dependent variable was obtained by an improved SZZ (Sliwerski, Zimmermann, and Zeller) algorithm (Kim et al. 2006). As a result, the discovered crash-inducing changes be incomplete or imprecise. This is a potential threat to the construct validity of the dependent variable, which needs to be eliminated in the future work.

The independent variables used in this study are the change-level features. They have a clear definition and can be easily collected. However, a possible threat is the bias from code implementation. In our study, we directly used the data sets shared online by Wu et al. We find that, for few candidate changes, all the change-level features have a value of 0. For example, for the NetBeans 6.5 data set, there are 1158 candidate changes of Form3 in total. Of the 1158 changes, 37 changes are the "zero-value-feature" changes. In our experiment, consistent with Wu et al.'s study, we excluded these changes for further analysis. Since only the compiled bytecode of ChangeLocator is shared, we are unable to examine the code for collecting the change-level features. Nonetheless, this threat needs to be reduced in the future work.

9.2 Internal validity

The first threat is the unknown effect of the employed modeling technique in ChangeLocator. In our experiment, the baseline approach ChangeLocator employed logistic regression as the modeling technique. This is the recommend modeling technique for ChangeLocator in Wu et al.'s study (Wu et al. 2018). Indeed, Wu et al. analyzed the influence of different modeling techniques (i.e. logistic regression, Naïve Bayes, BayesNet, and J48) on the effectiveness of ChangeLocator. Consequently, they found that logistic regression and J48 were the preferred

modeling techniques in ChangeLocator. In particular, they reported that J48 had a slight higher Recall@1 of 0.477 than logistic regression. However, ChangeLocator based on J48 is still worse than ChangeRanker in terms of Recall@1 (the observations from MRR and MAP are similar). Nonetheless, this threat needs to be reduced by investigating more modeling techniques in ChangeLocator in the future work.

The second threat is the unknown effect of the actual operation process of the subject project. According to (Wu et al. 2018), crash-inducing changes are determined by applying an improved SZZ algorithm (Kim et al. 2006) to mine crash-fixing changes. It is possible that some functions have no crash-inducing defects simply because they do not undergo frequent use under different operation environments. Consequently, some crash-inducing changes may be incorrectly labeled as non-crash-inducing changes, which may bias our finding. In other words, the actual operation process may have a confounding effect on the labels of candidate changes and hence affect our finding. This threat needs to be mitigated by controlled experiments in the future work.

9.3 External validity

The most important threat is that our findings may not be generalized to other projects. In our experiment, we used the NetBeans project used in Wu et al.'s study and we collect three new projects to compare the effectiveness between ChangeRanker and ChangeLocator. However, these data sets are somewhat scanty. As stated in (Wu et al. 2018), although there are many open-source projects available, it is not easy to collect a large amount of quality crash report data to conduct the experiment. The reason is that "crash stacks in bug reports are optional and not many users report the crash stacks manually" (Wu et al. 2018; An and Khomh 2015). Nonetheless, to mitigate this threat, there is a need to replicate our study across a wider variety of projects in the future.

10 Conclusion and future work

In this paper, we propose ChangeRanker, an approach boosting crash-inducing change localization with rank-performance-based feature subset selection. Based on the six releases of the NetBeans project, our experimental results show that, ChangeRanker can locate the crash-inducing changes for 56.4% of crash buckets by examining only top 1 change candidate. In addition, ChangeRanker achieves an MRR of 0.641 and an MAP of 0.610. Based on three new projects, ChangeRanker can achieve 0.323, 0.410 and 0.384 in terms of Recall@1, MRR and MAP in locating defect-inducing changes. Compared with the state-of-the-art approach ChangeLocator, these results correspond to a large improvement, regardless of whether Recall@1, MAP, or MRR is considered. This indicates that the generalization of our approach is excellent. In particular, when applied to a target crash bucket, ChangeRanker need fewer features to build the prediction model. Compared with other feature selection approaches, Rfs has more advantages in localizing crash-inducing changes. In summary, ChangeRanker provides an easy-to-use approach to locating crash-inducing changes.

In the future work, on the one hand, we plan to use more open-source projects to evaluate the effectiveness of ChangeRanker. This enables us to know to what extent our finding in this paper can be generalized to the other projects. On the other hand, there is a need to put it into the real world and let it be used by developers who work on a live code base. This enable us to examine whether ChangeRanker is practically important.

Acknowledgements This work is partially supported by the National Key R&D Program of China (2018YFB1003901) and the National Natural Science Foundation of China (61772259, 61872177, 61832009, 61772263).

References

P.D. Allison. Logistic regression using the SAS system: theory and application. 1999

An L, Khomh F (2015) An empirical study of crash-inducing commits in Mozilla Firefox. PROMISE:1–10

Benavoli A, Corani G, Mangili F (2016) Should we really use post-hoc tests based on mean-ranks? J Mach Learn Res 17(5):1–10

Benjamini Y, Hochberg Y (1995) Controlling the false discovery rate: a practical and powerful approach to multiple testing. J R Stat Soc Ser B 57:289–300

Bettenburg N, Premraj R, Zimmermann T (2008) Extracting structural information from bug reports. MSR:27–30

Chawla N, Bowyer K, Hall L, Kegelmeyer P (2002) Smote: synthetic minority over-sampling technique. J Artif Intell Res 16:321–357

Cohen J, Cohen P, West SG (2003) Applied multiple regression/correlation analysis for behavioral sciences. Lawrence Erlbaum, Hillsdale, NJ

Costa DA, Mcintosh S, Shang W (2017) A framework for evaluating the results of the SZZ approach for identifying bug-introducing changes. IEEE Trans Softw Eng 43(7):641–657

Dang Y, Wu R, Zhang H (2012) Rebucket: a method for clustering duplicate crash reports based on call stack similarity. ICSE:1084–1093

Gao K, Khoshgoftaar TM, Seliya N (2012) Predicting high-risk program modules by selecting the right software measurements. Softw Qual J 20(1):3–42

Ghotra B, McIntosh S, Hassan AE (2017) A large-scale study of the impact of feature selection techniques on defect classification models. MSR:146–157

Green SB (1991) How many subjects does it take to do a regression analysis? Multivar Behav Res 26(3):499–510

Guyon I, Elisseeff A (2003) An introduction to variable and feature selection. J Mach Learn Res 3:1157–1182

Hall M, Holmes G (2003) Benchmarking attribute selection techniques for discrete class data mining. IEEE Trans Knowl Data Eng 15(6):1437–1447

Hoang VD, Oentaryo RJ, Le (2018) TDB Network-clustered multi-modal bug localization. IEEE Transactions on Software Engineering, to appear

Holmes G, Donkin A, Witten IH (2002) WEKA: a machine learning workbench. ANZIIS:357–361

Jacek S, Zimmermann T, Zeller A (2005) When do changes induce fixes? MSR:1–5

Kim S, Zimmermann T, Pan K (2006) Automatic identification of bug-introducing changes. ASE:81–90

Kim S, Zimmermann T, Whitehead EJ (2007) Predicting faults from cached history. ICSE:489–498

Kim S, Whitehead EJ, Zhang Y (2008) Classifying software changes: clean or buggy? IEEE Trans Softw Eng 34(2):181–196

Kim D, Wang X, Kim S (2011) Which crashes should I fix first?: predicting top crashes at an early stage to prioritize debugging efforts. IEEE Trans Softw Eng 37(3):430–447

Kinshumann K, Glerum K, Greenberg S, Aul G, Orgovan V, Nichols G, Grant D, Loihle G, Hunt G (2011) Debugging in the (very) large: ten years of implementation and experience. Commun ACM 54(7):111–116

Kohavi R. R. A study of cross-validation and bootstrap for accuracy estimation and model selection. IJCAI 1995: 1137–1143

Kohavi R, John G (1998) The wrapper approach. Feature Extraction, Construction and Selection. Springer US

Le TDB, Oentaryo RJ, Lo D (2015) Information retrieval and spectrum based bug localization: better together. FSE:579–590

Lewis C, Lin Z, Sadowski C et al (2013) Does bug prediction support human developers? Findings from a google case study. ICSE:372–381

Lin D, Lin F, Lv Y, Cai F, Cao D (2018) Chinese character CAPTCHA recognition and performance estimation via deep neural network. Neurocomputing 288:11–19

Lukins SK, Kraft NA, Etzkorn LH (2008) Source code retrieval for bug localization using latent dirichlet allocation. WCRE:155–164

Manning CD, Raghavan P, Schütze H (2008) Introduction to information retrieval. Cambridge University Press

Moreno L, Bandara W, Haiduc S (2013) On the relationship between the vocabulary of bug reports and source code. ICSM:452–455

Moser R, Pedrycz W, Succi G (2009) A comparative analysis of the efficiency of change metrics and static code attributes for defect prediction. ICSE:181–190

Muthukumaran K, Rallapalli A, Murthy NLB (2015) Impact of feature selection techniques on bug prediction models. ISEC:120–129
Nguyen AT, Nguyen TT, Al-Kofahi J (2011) A topic-based approach for narrowing the search space of buggy files from a bug report. ASE:263–272
Rahman S, Ganguly KK, Sakib K (2016) An improved bug localization using structured information retrieval and version history. ICCIT
Rao S, Kak A (2011) Retrieval from software libraries for bug localization: a comparative study of generic and composite text models. MSR:43–52
Rathore SS, Gupta A (2014) A comparative study of feature-ranking and feature-subset selection techniques for improved fault prediction. ISEC
Rodríguez D, Ruiz R, Cuadrado-Gallego J, AguilarRuiz J (2007) Detecting fault modules applying feature selection to classifiers. IEEE Int Conf Inf Reuse Integr:667–672
Shaffer J (1986) Modified sequentially rejective multiple test procedures. J Am Stat Assoc 81(395):826–831
Song Q, Guo Y, Shepperd M (2018) A comprehensive investigation of the role of imbalanced learning for software defect prediction. IEEE Trans Softw Eng
Tantithamthavorn C, Mclntosh S, Hassan AE, Matsumoto K (2019) The impact of automated parameter optimization on defect prediction models. IEEE Trans Softw Eng 45(7):683–711
Uneno Y, Mizuno O, Choi EH (2016) Using a distributed representation of words in localizing relevant files for bug reports. QRS:183–190
Venkatesh GA (1991) The semantic approach to program slicing. PLDI:107–119
Wang S, Yao X (2009) Diversity analysis on imbalanced data sets by using ensemble models. CIDM:324–331
Wang S, Lo D, Lawall J (2014) Compositional vector space models for improved bug localization. ICSME:171–180
Wen M, Wu R, Cheung SC (2016) Locus: locating bugs from software changes. ASE:262–273
Wilcoxon F (1945) Individual comparisons by ranking methods. Biom Bull 1(6):80–83
Williams C, Spacco J (2008) SZZ revisited: verifying when changes induce fixes. DEFECTS:32–36
Wilson CR, Voorhis V, Morgan BL (2007) Understanding power and rules of thumb for determining sample size. Tutor Quant Methods Psychol 3(2):43–50
Wu R, Zhang H, Cheung SC (2014) Crashlocator: locating crashing faults based on crash stacks. ISSTA:204–214
Wu R, Wen M, Cheung SC (2018) Changelocator: locate crash-inducing changes based on crash reports. Empir Softw Eng 23(5):2866–2900
Xu Z, Liu J, Yang Z, An G, Jia X (2016) The impact of feature selection on defect prediction performance: An empirical comparison. ISSRE:309–320
Youm KC, Ahn J, Kim J (2016) Bug localization based on code change histories and bug reports. APSEC:190–197
Zhang L, Kim M, Khurshid S (2011) Localizing failure-inducing program edits based on spectrum information. ICSM:23–32

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Zhaoqiang Guo is a Ph.D. student in the Department of Computer Science and Technology at Nanjing University. His current research direction is bug localization and software metrics.

Yanhui Li is a researcher in the Department of Computer Science and Technology at Nanjing University. His current research direction is empirical software engineering.

Wanwangying Ma is a Ph.D. student in the Department of Computer Science and Technology at Nanjing University. Her research interests include program analysis and software metrics.

Yuming Zhou is a professor in the Department of Couputer Science and Technology at Nanjing University. His main research interests are empirical software engineering.

Hongmin Lu received the Ph.D. degree in computer science from Southeast University in 2013. She is a researcher in the Department of Computer Science and Technology at Nanjing University. Her research interests are software metrics and software maintenance.

Lin Chen received the Ph.D. degree in computer science from Southeast University in 2009. He is currently an associate professor in the Department of Computer Science and Technology at Nanjing University. His research interests are software analysis and software maintenance.

Baowen Xu received the BS, MS, and Ph.D. degrees in computer science from Wuhan University, Huazhong University of Science and Technology, and Beihang University, respectively. He is currently a professor in the Department of Computer Science and Technology at Nanjing University. His main research interests are programming languages, software testing, software maintenance, and software metrics. He is a member of the IEEE and the IEEE Computer Society.

Affiliations

Zhaoqiang Guo[1] · Yanhui Li[1] · Wanwangying Ma[1] · Yuming Zhou[1] · Hongmin Lu[1] · Lin Chen[1] · Baowen Xu[1]

Zhaoqiang Guo
naplus@smail.nju.edu.cn

Yanhui Li
yanhuili@nju.edu.cn

Wanwangying Ma
xiaobena@live.cn

Hongmin Lu
hmlu@nju.edu.cn

Lin Chen
lchen@nju.edu.cn

Baowen Xu
bwxu@nju.edu.cn

[1] State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China